INTERVIEWHUB · INTERVIEW PREP

Helm & Kubernetes Packaging — InterviewHub

Platform, SRE & DevOps interview guide

Questions	**100 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/helm_interview_guide.html**
Printed	**2026-10-23**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What is Helm and what problems does it solve in Kubernetes?

Core concepts

🎙 Spoken answer · ~68 s

Helm is the package manager for Kubernetes. Applications usually need many manifests: deployments, services, config maps, secrets, ingress, service accounts and more, and managing them as loose YAML files across environments becomes repetitive and error-prone. Helm packages them into a chart, a versioned bundle of templates and default values. At install time, Helm renders the templates with supplied values into plain manifests and sends them to the cluster through the Kubernetes API, recording the result as a release. This gives parameterisation for different environments, versioning of the application package, a single command to install, upgrade or roll back as a unit, and a way to share applications through repositories or OCI registries. Helm 3 is client-only, with no Tiller component in the cluster, and stores release state as secrets in the namespace. It does not replace understanding Kubernetes, and for simple cases plain manifests or Kustomize may be enough, but Helm is very widely used for third-party software and for reusable internal application packaging.

Likely follow-ups

- Which problems do templates and values address?
- How does Helm 3 differ from Helm 2?

Helm packages Kubernetes manifests into versioned charts with templates and values, renders them per environment and manages install, upgrade and rollback as releases. Helm 3 is client-only and stores release state as namespace secrets.

```
helm install myapp ./mychart -f values-prod.yaml
```

```
helm list -A
```

Interview tip: Say chart plus values equals release.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q2 Explain the core Helm concepts: chart, release, repository and values.

Core concepts

🎙 **Spoken answer · ~68 s**

A chart is the package: a directory or archive containing Chart.yaml metadata, default values, templates, and optional dependencies, documentation and schema. A release is a running instance of a chart in a cluster, with a name and namespace, and one chart can be installed many times as different releases, for example one per environment or tenant. Each upgrade creates a new numbered revision of the release, which is what makes history and rollback possible. A repository is a place where charts are published and discovered, either a classic HTTP server with an index file, or increasingly an OCI registry. Values are the configuration inputs: the chart's values.yaml supplies defaults, and users override them with files or command-line flags, and the merged result is used when rendering templates. Helm stores each revision's rendered manifest and values as a release record, so it can compute differences and revert. Understanding these four terms and how they relate explains most Helm commands and the usual workflow.

Likely follow-ups

- Can one chart have multiple releases?
- What is a revision?

A chart is the package, a release is an installed instance with numbered revisions, a repository publishes charts through HTTP index or OCI, and values are configuration merged over chart defaults at render time.

```
helm history myapp
```

```
helm get values myapp
```

Interview tip: Say one chart, many releases, each upgrade a revision.

📚 **Official docs**

- Helm: Values
- Helm: Chart repository guide
- Helm documentation

certs.eknathalabs.com

Q3 What are the essential Helm commands you use daily?

Core concepts

🎙 Spoken answer · ~72 s

For finding and fetching charts, helm search repo and helm show values reveal available charts and their options, and helm pull downloads one. To inspect before deploying, helm template renders manifests locally without a cluster, helm lint checks the chart for problems, and helm install with the dry-run flag simulates against the API server. To deploy, helm install creates a release, and helm upgrade with the install flag, often used in pipelines, installs or upgrades in one step. For operations, helm list shows releases, helm status shows details and notes, helm get values, manifest and hooks show what was applied, helm history lists revisions, helm rollback reverts to an earlier one, and helm uninstall removes a release. For authoring, helm create scaffolds a chart, helm package builds an archive, helm dependency update fetches subcharts, and helm push or helm registry login work with OCI registries. The helm diff plugin is widely used to preview changes before upgrades, and a typical flow is lint, template, diff, then upgrade with atomic and wait flags.

Likely follow-ups

- What does helm template do and not do?
- Which flags make upgrades safer?

Daily commands are search, show, template, lint, install, upgrade --install, list, status, get, history, rollback, uninstall, plus create, package and dependency update for authoring. Use the diff plugin and atomic and wait flags for safer upgrades.

```
helm upgrade --install myapp ./chart -n prod --create-namespace --atomic --wait
```

```
helm template myapp ./chart -f values.yaml | kubectl apply --dry-run=server -f -
```

Interview tip: Say lint, template, diff, then upgrade --install --atomic.

📚 Official docs

- Helm: Using Helm / commands
- Helm documentation

certs.eknathalabs.com

Q4 How does helm install work internally, from command to running resources?

Core concepts

🎙 **Spoken answer · ~71 s**

When you run helm install, the client first locates the chart, from a directory, archive, repository or OCI reference, and loads it, including any dependencies. It merges values in order of precedence: chart defaults, then values files in the order given, then command-line set flags. It then renders all templates with the Go template engine, using the merged values and built-in objects such as release name, namespace and chart metadata, producing Kubernetes manifests. The rendered output is validated, including against a values schema if present, and sorted into an install order by resource kind, so namespaces and custom resource definitions come before the resources that need them. Helm records the release in the cluster as a secret in the target namespace, then submits the resources to the Kubernetes API, running hooks at their defined points, such as pre-install jobs. With the wait flag it polls until resources are ready. Finally it prints the release status and any notes. Nothing runs server-side except Kubernetes itself, because Helm 3 has no controller.

Likely follow-ups

- Where is release information stored?
- In what order are resources installed?

Helm loads the chart, merges values, renders Go templates to manifests, validates them, orders resources by kind, stores the release record as a namespace secret, applies resources through the Kubernetes API with hooks, optionally waits for readiness and prints notes.

```
kubectl get secrets -n prod -l owner=helm
```

```
helm install myapp ./chart --debug --dry-run
```

Interview tip: Say render client-side, store release as a secret, apply via API.

📚 **Official docs**

- Helm: Using Helm / commands
- Helm documentation

certs.eknathalabs.com

Q5 What is the difference between helm install, helm upgrade and helm upgrade --install?

Core concepts

🎙 **Spoken answer · ~67 s**

Helm install creates a new release and fails if a release with that name already exists in the namespace. Helm upgrade modifies an existing release to a new chart version or new values, creating a new revision, and fails if the release does not exist. Helm upgrade with the install flag combines them: it installs when no release exists and upgrades when one does, which makes it idempotent and the usual choice in automation, so a pipeline does not need to check existence first. During an upgrade, Helm computes a three-way merge between the previous manifest, the live cluster state and the new manifest, so changes made manually in the cluster can be preserved or reconciled according to the strategy. Values are not remembered automatically unless asked, so reuse-values or reset-values flags matter, and I prefer to supply the complete values file each time for reproducibility. Useful flags include atomic for automatic rollback on failure, wait with a timeout, and create-namespace.

Likely follow-ups

- Why is upgrade --install preferred in pipelines?
- What does the atomic flag do?

Install creates a release and fails if it exists; upgrade changes an existing release and creates a revision; upgrade --install does either, so it is idempotent for pipelines. Supply complete values each time and use atomic and wait.

```
helm upgrade --install myapp oci://registry/charts/myapp --version 1.4.2 -f prod.yaml --
atomic --timeout 5m
```

Interview tip: Say upgrade --install with full values each run.

📚 **Official docs**

- Helm: Chart hooks
- Helm: Plugins
- Helm: Upgrade

certs.eknathalabs.com

Q6 How does Helm store release state, and what happened to Tiller? Core concepts

🎙 Spoken answer · ~65 s

In Helm 2, a server component called Tiller ran in the cluster with broad privileges and stored release data in config maps. That created security concerns, since anyone who could reach Tiller could effectively act with its permissions, and it complicated role-based access control. Helm 3 removed Tiller completely. The client now talks directly to the Kubernetes API using the user's kubeconfig credentials, so permissions are those of the person or pipeline running Helm and standard RBAC applies. Release state is stored as Kubernetes secrets in the release's namespace, by default one secret per revision named sh.helm.release.v1.name.vN, containing a compressed, encoded record of the chart, values, rendered manifest and status. Releases are therefore namespaced, and the same release name can be used in different namespaces. This design also makes backup and cleanup straightforward, and the history limit setting can cap how many revisions are kept. Because these secrets contain values, access to them should be restricted.

Likely follow-ups

- Why was Tiller a security concern?
- How do you limit stored revisions?

Helm 2 used a privileged in-cluster Tiller; Helm 3 removed it, acts with the caller's RBAC, and stores each revision as a secret named sh.helm.release.v1 in the release namespace. Restrict access to those secrets since they contain values.

```
kubectl get secret sh.helm.release.v1.myapp.v3 -n prod
```

```
helm upgrade myapp ./chart --history-max 10
```

Interview tip: Say no Tiller, state in namespace secrets.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q7 When would you use Helm compared with Kustomize or plain manifests? Core concepts

🎙 Spoken answer · ~66 s

Plain manifests are the simplest and most transparent, fine for small, stable deployments, but they duplicate across environments. Kustomize works on plain YAML without templating, using bases and overlays with patches, so it is declarative, built into kubectl and easy to review, and suits customising your own manifests per environment. Helm adds a templating language, packaging, versioning, dependency management, release tracking with rollback, hooks and distribution through repositories, which is valuable when you consume third-party software, need to ship an application to many users with configurable options, or want release lifecycle management. Its drawbacks are template complexity, YAML-in-templates readability and a learning curve. The approaches combine well: render Helm charts and then post-process with Kustomize, or use Helm to install vendor charts and Kustomize for your own overlays, and GitOps controllers like Argo CD and Flux support both. My rule is to use Helm for packaged, parameterised or third-party applications and Kustomize for simple environment variations, avoiding unnecessary templating.

Likely follow-ups

- Can Helm and Kustomize be combined?
- What are the downsides of Helm templating?

Plain manifests suit small static setups, Kustomize gives patch-based overlays without templating, and Helm adds packaging, versioning, dependencies, hooks and release management. Use Helm for third-party or configurable apps, Kustomize for simple variants, or combine them.

```
helm template myapp ./chart | kubectl kustomize
```

```
kubectl apply -k overlays/prod
```

Interview tip: Say Helm packages and versions; Kustomize patches.

📚 Official docs

- Kusto Query Language (KQL)
- Helm: Chart template guide
- Helm: Post-rendering

certs.eknathalabs.com

Q8 How do you install and configure Helm, and manage Kubernetes context and namespaces?

Core concepts

🎙 Spoken answer · ~69 s

Helm is a single binary installed through a package manager, a script or a release download, with the version pinned in CI for reproducibility. It uses the current kubeconfig context, from the KUBECONFIG variable or the default file, so before running commands I verify the context and namespace, because installing into the wrong cluster is a classic mistake. The kube-context and namespace flags select the target explicitly, and pipelines should always pass them. Releases are namespaced, and the create-namespace flag makes the namespace if absent, though governance may require creating namespaces with quotas and policies separately. Helm reads repositories from a configuration file, caches indexes and charts, and its paths are controlled by environment variables for config, cache and data directories, useful in CI. For OCI registries, helm registry login stores credentials. Shell completion and a plugin set such as diff help day-to-day. Permissions derive from the kubeconfig identity, so service accounts for CI should have least-privilege roles scoped to the namespaces they deploy to.

Likely follow-ups

- Why pass context and namespace explicitly in CI?
- Where does Helm keep repository config?

Install a pinned Helm binary, verify kubeconfig context and namespace before acting, pass kube-context and namespace explicitly in automation, use create-namespace carefully, configure repositories or OCI logins and give CI least-privilege access.

```
helm version
```

```
kubectl config current-context
```

```
helm list -n prod --kube-context prod-eu
```

Interview tip: Say always pass context and namespace in pipelines.

📚 Official docs

- Helm: Using Helm / commands
- Helm documentation

certs.eknathalabs.com

Q9 What does helm uninstall remove, and what can be left behind?

Core concepts

🎙 Spoken answer · ~69 s

Helm uninstall deletes the resources that were created from the release's manifest, executes pre-delete and post-delete hooks, and by default removes the release history records, though keep-history retains them so you could see or roll back to past revisions. Things that can remain include persistent volume claims created by stateful sets through volume claim templates, since Kubernetes does not delete them when the stateful set goes, so data survives, which is usually desirable but leaves storage cost; custom resource definitions installed from a chart's crds directory, which Helm intentionally never deletes because removing them would delete every custom resource of that type; resources annotated with the helm resource policy keep; namespaces that Helm did not create or that were made with the create-namespace flag; and objects created by hooks or by controllers at runtime, such as cloud load balancers that depend on service finalisers. After uninstalling, I review leftover volumes, secrets and CRDs deliberately and clean them up with explicit commands, to avoid accidental data loss.

Likely follow-ups

- Why does Helm not delete CRDs?
- How do you keep a resource on uninstall?

Uninstall removes manifest resources and runs delete hooks, optionally keeping history. PVCs from stateful sets, CRDs, resources with the keep policy, namespaces and runtime-created objects can remain, so review and clean them explicitly.

```
helm uninstall myapp -n prod --keep-history
```

```
kubectl get pvc,crd -n prod
```

Interview tip: Say PVCs and CRDs survive uninstall by design.

📚 Official docs

- Helm: Using Helm / commands
- Helm documentation

certs.eknathalabs.com

Q10 What are Helm plugins and which ones are useful?

Core concepts

🎙 Spoken answer · ~68 s

Plugins extend the Helm client with extra subcommands. They are installed from a repository URL and live in the Helm data directory, each described by a plugin manifest that defines the command and hooks. Because they run locally with your permissions, I treat them as software supply chain items: install from trusted sources, pin versions and verify them where possible. Widely used ones include helm-diff, which shows what an upgrade would change in the cluster and is the most valuable one for review in pipelines; helm-secrets, which integrates with SOPS to encrypt values files; helm-unittest, which lets you write unit tests for templates; helm-docs, which generates chart documentation from values comments; and the helm-mapkubeapis plugin, which updates stored release metadata when Kubernetes removes deprecated APIs. Others add S3 or Git-based repository support or conversion helpers. In CI, I pin plugin versions in a bootstrap step so builds are reproducible. Since the OCI registry support matured, some older repository plugins are less needed.

Likely follow-ups

- What does helm diff add?
- Why pin plugin versions?

Plugins add Helm subcommands; install from trusted sources and pin versions. Useful ones are helm-diff for previewing upgrades, helm-secrets with SOPS, helm-unittest for template tests, helm-docs for documentation and mapkubeapis for removed APIs.

```
helm plugin install https://github.com/databus23/helm-diff --version 3.9.0
```

```
helm diff upgrade myapp ./chart -f prod.yaml
```

Interview tip: Say helm diff before every production upgrade.

📚 Official docs

- Helm: Plugins
- Helm documentation

certs.eknathalabs.com

Q11 How does Helm templating work, and what are the built-in objects?

Templating & Go

🎙 Spoken answer · ~71 s

Helm uses Go's text template engine, extended with the Sprig function library and Helm-specific functions. Template files in the templates directory contain YAML with actions in double curly braces that insert values, call functions and control flow. Rendering produces plain manifests. Built-in objects are available at the top-level dot: Values holds the merged configuration; Release gives the name, namespace, revision, and whether this is an install or an upgrade; Chart exposes the content of Chart.yaml; Files gives access to non-template files in the chart; Capabilities reports Kubernetes version and available API versions, useful for compatibility branches; and Template holds the current template name and path. Subchart values are scoped to the subchart under its own name. Whitespace control with dashes inside braces trims surrounding lines, which matters because YAML is indentation-sensitive. Files starting with an underscore, such as the helpers file, are not rendered as manifests and are meant for named templates. The notes file prints usage instructions after install. Developing templates relies heavily on the template and debug commands.

Likely follow-ups

- What does Capabilities provide?
- Why are whitespace dashes important?

Helm renders Go templates with Sprig functions, exposing Values, Release, Chart, Files, Capabilities and Template objects. Use whitespace control for valid YAML, underscore files for helpers and template with debug to inspect output.

```
metadata:
  name: {{ .Release.Name }}-web
  namespace: {{ .Release.Namespace }}
```

```
helm template myapp ./chart --debug
```

Interview tip: Say the dot is context; Values, Release and Chart are its parts.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q12 How do you use conditionals and loops in Helm templates?

Templating & Go

🎙 Spoken answer · ~72 s

Conditionals use if, else if, else and end. A value is false when it is boolean false, zero, an empty string, an empty list or map, or nil, which makes it easy to guard optional blocks such as an ingress that renders only when enabled. Loops use range, which iterates over lists and maps; inside the loop the dot is rebound to the current element, so to reach top-level objects such as Release or Values you use the dollar sign, which always refers to the root context. For maps, range provides key and value variables. The with action sets the dot to a value and skips the block if it is empty, which is handy for optional nested sections, but has the same rebinding trap. Variables can be assigned with colon-equals to keep references. Common patterns are rendering a list of environment variables from a map, creating several ingress hosts, or toggling resources. Care with indentation is required, using nindent or indent to place output correctly, and whitespace trimming to avoid blank lines.

Likely follow-ups

- Why use the dollar sign inside range?
- What counts as false in a condition?

Use if/else for optional blocks, noting empty values are false; range for lists and maps, where dot is rebound so use $ for the root; with for optional nested values; and nindent and whitespace trimming to keep YAML valid.

```
{{- if .Values.ingress.enabled }}
{{- range .Values.ingress.hosts }}
- host: {{ . | quote }}
{{- end }}
{{- end }}
```

Interview tip: Say $ is the root inside range and with.

📚 Official docs

- Helm: Chart best practices
- Helm: Chart template guide
- Helm: Using Helm / commands

certs.eknathalabs.com

Q13 What are named templates and _helpers.tpl, and how do include and template differ?

Templating & Go

🎙 **Spoken answer · ~68 s**

Named templates are reusable template fragments defined with the define action and a name, conventionally placed in _helpers.tpl, which Helm does not render as a manifest because of the underscore. Typical helpers produce the chart's full name, the standard label set, the selector labels and the service account name. They are called with either template or include. The template action inserts the output directly and cannot be piped to other functions, whereas include is a function returning a string, so its output can be passed through filters such as indent, nindent, quote or sha256sum, which is why include is preferred nearly always. Both need the context passed explicitly, usually the dot or dollar sign, because a defined template does not inherit the caller's context automatically. Names are global across a chart and its subcharts, so they should be namespaced with the chart name as a prefix to avoid collisions. Centralising labels and names in helpers keeps resources consistent and makes changes in one place.

Likely follow-ups

- Why is include better than template?
- Why prefix helper names with the chart name?

Define reusable fragments with define in _helpers.tpl and call them with include, which returns a string that can be piped to indent or nindent; template cannot be piped. Pass context explicitly and prefix names with the chart name since they are global.

```
{{- define "mychart.labels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
{{- end }}
```

```
labels:
  {{- include "mychart.labels" . | nindent 4 }}
```

Interview tip: Say include plus nindent, never template for helpers.

📚 **Official docs**

- Helm: Chart template guide
- Helm: Plugins
- Helm: Using Helm / commands

certs.eknathalabs.com

Q14 Which Sprig and Helm template functions do you use most, such as default, required, quote, toYaml and tpl?

Templating & Go

🎙 Spoken answer · ~70 s

The default function supplies a fallback when a value is empty, keeping templates tolerant of missing configuration. The required function fails rendering with a clear message when a mandatory value is not provided, which is better than producing a broken manifest. Quote ensures strings are quoted, avoiding YAML type surprises such as numbers or yes and no being interpreted as other types. The toYaml function converts a map or list from values into YAML text, typically piped to nindent so entire blocks such as resources, tolerations or affinity can be passed through without the template knowing their fields. Tpl evaluates a string value as a template, letting users write expressions inside values, such as names that reference the release, at the cost of complexity and the risk of surprising behaviour. Others I use are trunc and trimSuffix for the 63-character name limits, sha256sum for config checksums that trigger rolling restarts, lookup to read existing cluster objects at install time, b64enc for secrets, and ternary for compact conditionals.

Likely follow-ups

- What does required do and why use it?
- What is tpl used for?

Common functions are default for fallbacks, required for mandatory values, quote for safe strings, toYaml with nindent for pass-through blocks, tpl for templated values, trunc for name limits, sha256sum for checksums and lookup for cluster reads.

```
image: {{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}
```

```
apiUrl: {{ required "apiUrl is required" .Values.apiUrl | quote }}
```

```
resources:
  {{- toYaml .Values.resources | nindent 2 }}
```

Interview tip: Say required for mandatory values, toYaml for blocks.

📚 Official docs

- Helm: Chart best practices
- Helm: Chart template guide
- Helm: Using Helm / commands

certs.eknathalabs.com

Q15 How do you trigger a rolling restart of pods when a ConfigMap or Secret changes?

Templating & Go

🎙 Spoken answer · ~71 s

Kubernetes does not restart pods when a config map or secret that they consume as environment variables changes, and mounted files update eventually but the application may not reload them. The common Helm pattern is to add an annotation to the pod template containing a checksum of the rendered config map or secret, computed in the template with include to render that file, then sha256sum. When the config content changes, the checksum changes, the pod template differs, and the deployment controller performs a normal rolling update. This keeps the restart controlled and respects rollout strategy and disruption budgets. The annotation needs the file path of the template in the include call, and it should be computed from the rendered output so any change in values flows through. Alternatives include controllers such as Reloader, which watch for changes and trigger rollouts, or applications that watch files and reload themselves. For secrets managed externally, the checksum approach cannot see the change, so a reloader or an explicit version annotation in values is used.

Likely follow-ups

- Why don't pods restart automatically on config change?
- What alternative exists to checksum annotations?

Add a pod template annotation with the sha256sum of the rendered ConfigMap or Secret so changes alter the pod template and trigger a rolling update. Alternatives are Reloader-style controllers or apps that reload files themselves.

```
spec:
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
sha256sum }}
```

Interview tip: Say checksum annotation to force a rolling update.

📚 Official docs

- Kubernetes: ConfigMaps
- External Secrets Operator

certs.eknathalabs.com

Q16 How do you debug a Helm template that renders the wrong output or fails?

Templating & Go

🎙 Spoken answer · ~71 s

I work from the outside in. Helm lint catches structural problems and common mistakes. Helm template renders locally without a cluster, and with the debug flag it prints even invalid YAML with the error context, so I can see where indentation or a missing value went wrong. The show-only flag renders a single file for focus. Helm install with dry-run and debug goes further by contacting the API server for validation and showing computed values. For specific values I temporarily print them using toYaml or toJson in a comment or a fail call, and checking the merged values with helm get values or the all flag. Common causes are indentation errors from missing nindent, the dot being rebound inside range or with, missing quote around values that look like numbers or booleans, nil pointer errors from accessing nested keys of missing maps, and whitespace dashes eating newlines. Piping the output through a YAML linter or kubeconform validates schema. A values schema file prevents many problems early by rejecting bad input.

Likely follow-ups

- What does --show-only do?
- What causes nil pointer errors?

Use helm lint, then helm template with --debug and --show-only to inspect output, dry-run for server-side validation, and print values with toYaml to check. Common causes are indentation, rebound dot, missing quotes, nil nested keys and whitespace trimming.

```
helm template myapp ./chart --debug --show-only templates/deployment.yaml
```

```
helm template myapp ./chart | kubeconform -strict -summary
```

Interview tip: Say template --debug, then kubeconform.

📚 Official docs

- Helm: Chart best practices
- Helm: Chart template guide
- Helm: Using Helm / commands

certs.eknathalabs.com

Q17 How do you handle optional blocks and nested values safely to avoid nil pointer errors?

Templating & Go

🎙 Spoken answer · ~73 s

Accessing a key under a missing parent map causes a nil pointer evaluating error, for example reading a nested setting when the parent block is absent from values. To avoid it, I make sure defaults exist in values.yaml for every parent map, so the structure is always present. In templates, I guard access with if or with, so the nested key is only read when the parent exists. The dig function from Sprig safely walks nested maps with a default, and the hasKey function checks for a key without triggering errors. Using the default function with an empty map and parentheses lets me chain safely. Optional features should be controlled by an enabled flag, so the template does not depend on the presence of other keys. A values schema marks required fields and types, catching mistakes before rendering. For lists, I check length before indexing. The goal is that a user supplying minimal values still gets a working release, while a missing mandatory value fails early with the required function and a clear message.

Likely follow-ups

- What does dig do?
- How does a schema help here?

Define defaults for every parent map in values.yaml, guard nested access with if, with or hasKey, use dig for safe traversal with a default, control features with enabled flags and fail early with required or a values schema.

```
{{ dig "service" "annotations" "team" "unknown" .Values }}
```

```
{{- if and .Values.ingress .Values.ingress.enabled }}
```

Interview tip: Say defaults for parents, dig for safe reads.

📚 Official docs

- Helm: Values

certs.eknathalabs.com

Q18 How do you write a chart that supports multiple Kubernetes API versions?

Templating & Go

🎙 Spoken answer · ~70 s

Kubernetes removes and promotes API versions over time, such as Ingress moving from extensions to networking.k8s.io v1beta1 and then v1, so a chart that targets many clusters needs to adapt. The Capabilities object lets a template check available versions: Capabilities.APIVersions.Has tests for a group version or kind, and Capabilities.KubeVersion exposes the cluster version for semver comparisons. A helper template can then choose the apiVersion string and adjust fields that differ between versions, such as the Ingress backend format and path type. The caveat is that helm template without a cluster uses default capabilities unless the api-versions and kube-version flags are given, so CI should pass those to test several targets. I also declare a kubeVersion constraint in Chart.yaml to state supported ranges, and drop support for very old versions on a published schedule rather than carrying branches forever. For releases already installed with removed APIs, the mapkubeapis plugin updates stored metadata so upgrades do not fail. Testing against multiple versions with kubeconform in a matrix catches regressions.

Likely follow-ups

- How does helm template know the cluster version offline?
- What does the kubeVersion field do?

Use Capabilities.APIVersions.Has and KubeVersion in helpers to pick apiVersion and fields, pass kube-version and api-versions flags when templating offline, declare kubeVersion in Chart.yaml, drop old versions on a schedule and use mapkubeapis for existing releases.

```
{{- if .Capabilities.APIVersions.Has "networking.k8s.io/v1/Ingress" }}
apiVersion: networking.k8s.io/v1
{{- end }}
```

```
helm template x ./chart --kube-version 1.29.0 --api-versions networking.k8s.io/v1/Ingress
```

Interview tip: Say Capabilities, plus CI across several versions.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q19 Why are Kubernetes resource names limited to 63 characters in charts, and how do you build safe names?

Templating & Go

🎙 **Spoken answer · ~71 s**

Many Kubernetes names, including services and labels values, follow DNS label rules with a maximum of 63 characters, and pod names generated from deployments add suffixes, so a long release name combined with a chart name can exceed limits and cause installs to fail. The standard scaffold from helm create defines a fullname helper that handles this: if the user supplies a full name override it is used; otherwise the release name and chart name are combined, avoiding duplication if the release name already contains the chart name, then truncated to 63 characters and trailing hyphens trimmed. Charts should use this helper consistently for all resource names and for selectors, with suffixes such as component names accounted for in the truncation length. Label values need the same care, and chart versions with plus signs must be converted for labels. Name overrides in values let users shorten. Selectors must stay stable across upgrades because deployment selectors are immutable, so changing how names are computed in a later chart version can break upgrades.

Likely follow-ups

- What does the fullname helper do?
- Why must selector labels stay stable?

DNS label rules cap many names at 63 characters, so use the fullname helper that combines release and chart names, truncates and trims hyphens, account for suffixes, provide name overrides and keep selectors stable since they are immutable.

```
{{- define "mychart.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end }}
```

Interview tip: Say fullname helper, truncate to 63, keep selectors stable.

📚 **Official docs**

- Helm documentation

certs.eknathalabs.com

Q20 How do you manage whitespace and indentation problems in Helm templates?

Templating & Go

🎙 Spoken answer · ~72 s

YAML depends on indentation, and Go templates leave the lines containing actions in the output, so unmanaged whitespace produces blank lines or misaligned blocks. A dash immediately inside the opening braces trims whitespace and newlines before the action, and a dash inside the closing braces trims after it. A typical block starts with an opening brace and dash so that control lines vanish from output. For inserting multi-line content such as the result of toYaml or include, the indent function adds spaces to every line, and nindent first adds a newline, which is the usual choice at the end of a key line. The number must match the nesting depth of the key. Mistakes appear as YAML parse errors, keys at the wrong level or, worse, silently valid but misplaced fields. I check output with helm template and a YAML linter, keep templates consistently formatted, avoid deeply nested logic inside blocks, and extract complicated parts to helpers. Editor support for Go templates in YAML is limited, so tests and rendered-output review are important.

Likely follow-ups

- What is the difference between indent and nindent?
- What does a dash in braces do?

Use dashes in braces to trim whitespace, nindent or indent to place multi-line output at the correct depth, check rendered output with helm template and a YAML linter and move complex logic into helpers.

```
{{- with .Values.nodeSelector }}
nodeSelector:
  {{- toYaml . | nindent 2 }}
{{- end }}
```

Interview tip: Say dash trims, nindent places.

📚 Official docs

- Helm: Chart best practices
- Helm: Chart template guide
- Helm: Using Helm / commands

certs.eknathalabs.com

Q21 How are Helm values merged, and what is the order of precedence?

Values & config

🎙 **Spoken answer · ~73 s**

Helm builds the final values from several sources with a defined order, where later sources override earlier ones. The base is the chart's own values.yaml. For subcharts, the parent chart's values under the subchart's name override the subchart defaults. Then files supplied with the values flag are applied in the order given, so a later file overrides an earlier one. Finally, individual overrides from the set, set-string and set-file flags win over everything. Maps are merged recursively key by key, so overriding one nested key leaves siblings intact, but lists are replaced entirely, not merged, which surprises people when overriding a single container argument or host entry. Setting a key to null removes the default. On upgrades, the reuse-values flag takes the previous release values as the base, while reset-values returns to chart defaults, and mixing them causes drift, so I prefer to pass a complete set of values files on every run, kept in version control. The values that were actually applied can be inspected afterwards with the get values command and the all flag.

Likely follow-ups

- Are lists merged or replaced?
- What does setting a key to null do?

Order is chart values.yaml, parent overrides for subcharts, values files in sequence, then set flags. Maps merge recursively but lists are replaced; null deletes a default. Prefer complete values files in Git over reuse-values.

```
helm upgrade --install app ./chart -f base.yaml -f prod.yaml --set image.tag=1.4.2
```

```
helm get values app --all
```

Interview tip: Say maps merge, lists replace.

📚 **Official docs**

- Helm: Chart best practices
- Helm: Values
- Helm documentation

certs.eknathalabs.com

Q22 How do you organise values files for multiple environments?

Values & config

🎙 Spoken answer · ~71 s

A common structure is a base values file holding shared settings plus one small overlay file per environment containing only the differences, applied in order so the environment file overrides the base. Per-region or per-cluster layers can be added the same way, giving a layered model such as defaults, environment, then cluster. Keeping files small and focused reduces duplication and makes reviews show only meaningful differences. The files live in version control beside the pipeline or in a GitOps repository, with the chart version pinned separately so a values change and a chart upgrade are distinct, reviewable events. Secrets are not stored in plain text there; they come from encrypted files, an external secrets operator or the pipeline. Naming conventions make the layering obvious. In GitOps tools such as Argo CD or Flux, the application definition lists the value files or inline values for each environment. A values schema validates all layers in CI, and a diff of rendered manifests between environments helps confirm that only expected differences exist before promotion.

Likely follow-ups

- Why keep chart version and values changes separate?
- How do you validate every environment's values?

Use a shared base values file plus small per-environment, region or cluster overlays applied in order, keep them in Git with the chart version pinned separately, store secrets elsewhere and validate all layers with a schema and rendered diffs in CI.

```
helm upgrade --install app ./chart -f values/base.yaml -f values/prod.yaml -f values/prod-eu.yaml
```

```
diff <(helm template app ./chart -f values/base.yaml -f values/stg.yaml) <(helm template app ./chart -f values/base.yaml -f values/prod.yaml)
```

Interview tip: Say base plus thin overlays, secrets elsewhere.

📚 Official docs

- Helm: Values

certs.eknathalabs.com

Q23 What is values.schema.json and why should every chart have one?

Values & config

🎙 Spoken answer · ~68 s

A values schema is a JSON Schema file placed at the root of the chart. Helm validates the merged values against it during install, upgrade, lint and template, and fails with a precise message if something violates it. It documents the expected structure, types, allowed enumerations, ranges, patterns and required properties, and it can set additionalProperties to false to catch typos in key names, which otherwise silently do nothing. Benefits include catching mistakes before anything reaches the cluster, giving users clear errors instead of confusing template failures, enabling editor auto-completion and validation, and serving as living documentation. Schemas apply to subcharts too, validated separately. Care is needed so the schema is not too strict, as that can break legitimate overrides, and it must be updated whenever values change. Tools such as helm-schema or generators can produce a first version from values.yaml and comments. Combined with the required function for cross-field rules that a schema cannot express, it makes charts much more robust for consumers.

Likely follow-ups

- What does additionalProperties false catch?
- When does Helm validate against the schema?

A values.schema.json defines types, enums, required fields and patterns; Helm validates merged values on install, upgrade, lint and template, catching typos and bad input early and documenting options. Keep it in sync with values.yaml.

```
{
 "$schema": "https://json-schema.org/draft-07/schema#",
 "type": "object",
 "required": ["image"],
 "properties": {"replicaCount": {"type": "integer", "minimum": 1}}
}
```

Interview tip: Say schema fails fast with a clear message.

📚 Official docs

- Helm: Values
- Helm documentation

certs.eknathalabs.com

Q24 What are the differences between --set, --set-string, --set-file and --set-json? Values & config

🎙 Spoken answer · ~74 s

The set flag assigns a value using dotted paths, and Helm infers types, so a value that looks like a number or boolean becomes one, and commas separate multiple pairs. Lists use index or brace syntax, and special characters must be escaped, which makes complex values awkward and error-prone. The set-string flag forces a string, which is needed when something like an image tag of 1.20 or an identifier of all digits would otherwise be converted into a number and lose meaning, or a true that must be text. The set-file flag reads the value from a file's content, useful for certificates or scripts. The set-json flag takes a JSON expression, handy for lists and nested structures without escaping troubles. For anything beyond simple scalars I prefer values files, which are reviewable, diffable and version-controlled, and keep set only for a few dynamic values injected by the pipeline such as the image tag. Never pass secrets through set in logs-visible commands, since they appear in process lists and CI output, and are stored in the release record anyway.

Likely follow-ups

- When do you need set-string?
- Why prefer values files over many set flags?

Set infers types; set-string forces strings for tag-like values; set-file loads file content; set-json passes structured JSON. Prefer values files for reviewability and reserve set for a few dynamic values like image tags, and never for secrets.

```
helm upgrade app ./chart --set-string image.tag=1.20 --set-file tls.crt=./tls.crt --set-json 'tolerations=[{"key":"gpu","operator":"Exists"}]'
```

Interview tip: Say set-string stops 1.20 becoming 1.2.

📚 Official docs

- Helm: Values
- Helm: Plugins

certs.eknathalabs.com

Q25 How do you design a good values.yaml for a chart others will consume?

Values & config

🎙 Spoken answer · ~70 s

A good values file is the chart's public interface, so I design it deliberately. Structure is logical and grouped by component or concern, such as image, service, ingress, resources, autoscaling and persistence, with consistent naming in camel case and flat enough to be convenient but nested where it helps. Every option has a safe default so the chart installs with no overrides in a development setting, and production-sensitive settings such as replicas, resources and persistence are easy to find. Comments document each key, with the helm-docs format generating a README table automatically. Booleans use an enabled convention for optional components. I expose standard hooks: extra environment variables, annotations, labels, volumes, node selectors, tolerations, affinity and security context, so users rarely need to fork. Secrets are referenced by existing secret name rather than embedded. I avoid exposing every field of every object, avoid deep nesting, keep names stable across versions as breaking changes need a major version, and add a schema. Examples of common production overrides are included.

Likely follow-ups

- How does helm-docs help?
- Why reference existing secrets instead of embedding?

Group values logically with consistent names, safe defaults, documented comments and helm-docs, enabled flags for optional parts, standard extension points like annotations and tolerations, secret references, stable names across versions and a schema.

```
image:
  repository: nginx
  tag: ""
resources: {}
autoscaling:
  enabled: false
```

```
helm-docs --chart-search-root .
```

Interview tip: Say values are the chart's API; version them like one.

📚 Official docs

- Helm: Values
- Helm: Provenance and integrity
- Helm documentation

certs.eknathalabs.com

Q26 How do global values work, and how do values flow to subcharts?

Values & config

🎙 **Spoken answer · ~70 s**

A subchart is a dependency chart stored in the charts directory, and it has its own values. A parent chart overrides a subchart's values by placing them in its own values file under a key equal to the subchart's name or alias; those settings are passed down and take precedence over the subchart defaults. The reverse is not true: a subchart cannot see the parent's values, which keeps charts independent. The exception is the special global key. Anything under global in the parent is made available to the parent and to every subchart, accessed as .Values.global, which suits shared settings such as image registry, environment name, storage class or domain. The import-values mechanism in the dependency definition can also pull selected values from a subchart up to the parent. Subcharts should work standalone, and global values should be optional with defaults. Overuse of global creates hidden coupling, so I keep it for genuinely cross-cutting settings and document it. Dependency conditions and tags enable or disable entire subcharts from values.

Likely follow-ups

- Can a subchart read parent values?
- What are conditions and tags for dependencies?

Parents override subchart values under the subchart's name or alias; subcharts cannot read parent values except those under the global key, shared with all. Use import-values for the reverse, conditions and tags to toggle subcharts, and keep global small.

```
postgresql:
  auth:
    database: app
global:
  imageRegistry: registry.example.com
```

Interview tip: Say down by name, shared by global.

📚 **Official docs**

- Helm: Values
- Helm: Charts and dependencies
- Helm documentation

certs.eknathalabs.com

Q27 How do you handle sensitive values in Helm without committing them in plain text?

Values & config

🎙 Spoken answer · ~70 s

Values files are stored in Git and in release records, so plain secrets there are a risk. Options start with avoiding the need: reference existing Kubernetes secrets by name and let a separate process create them, using the External Secrets Operator or the Secrets Store CSI driver to sync from Vault or a cloud secrets manager, so the chart never contains the secret. If values must carry secrets, encrypt the values files with SOPS using KMS, age or PGP keys, decrypted only at deploy time by the helm-secrets plugin or the GitOps tool, such as Flux's native SOPS support. Sealed Secrets lets you commit an encrypted secret that only the cluster can decrypt. In pipelines, secrets can be injected from the CI secret store at run time, though anything passed to Helm still ends up in the release secret in the cluster, so access to those must be restricted and encryption at rest enabled. Rotate credentials, never print values in logs, and scan repositories for leaked secrets.

Likely follow-ups

- Why is the release secret still a concern?
- What do External Secrets do?

Prefer referencing secrets created by External Secrets, CSI drivers or Sealed Secrets; otherwise encrypt values with SOPS and helm-secrets. Remember release records contain values, so restrict access, enable encryption at rest, rotate and scan for leaks.

```
sops --encrypt --age $AGE_PUB values-secret.yaml > values-secret.enc.yaml
```

```
helm secrets upgrade --install app ./chart -f values-secret.enc.yaml
```

Interview tip: Say reference secrets, don't embed them.

📚 Official docs

- Helm: Chart template guide
- Helm: Values
- Helm documentation

certs.eknathalabs.com

Q28 What are the problems with --reuse-values and how do you avoid configuration drift?

Values & config

🎙 Spoken answer · ~69 s

Reuse-values makes an upgrade start from the values of the previous release instead of the new chart's defaults, then apply any new overrides. The trap is that new keys added to the chart's values.yaml in a later chart version are not picked up, because the old computed values are reused, so templates may see missing values and fail or behave differently, and defaults changes are silently ignored. Over time the release's effective configuration diverges from what is in Git, and nobody can say precisely what is deployed. The remedy is a declarative approach: supply the full set of values files on every upgrade, treating Git as the source of truth, so the result is reproducible and an upgrade is deterministic. The reset-then-reuse-values option can reapply the stored values on top of new defaults when needed. Auditing is done with get values and the all flag, and diff plugin output compared with the repository. GitOps controllers enforce this by continuously reconciling from Git, removing manual drift.

Likely follow-ups

- Why are new chart defaults missed with reuse-values?
- How does GitOps remove drift?

Reuse-values keeps old computed values so new chart defaults are ignored and the release drifts from Git. Pass full values files every time, treat Git as truth, audit with get values and use GitOps reconciliation.

```
helm get values app --all -o yaml > live.yaml
```

```
helm upgrade app ./chart -f values/base.yaml -f values/prod.yaml
```

Interview tip: Say Git is the truth; avoid reuse-values.

📚 Official docs

- Infrastructure drift (Terraform: plan -refresh-only)
- Helm: Chart template guide
- Helm: Values

certs.eknathalabs.com

Q29 How do you pass complex structures such as lists of maps, multiline strings and typed numbers through values?

Values & config

🎙 Spoken answer · ~73 s

Complex data is easiest and safest in values files using native YAML, which supports lists, maps and block scalars with pipe for multiline text and keeps indentation visible. In templates, toYaml renders these structures into manifests, and tpl can evaluate a multiline string that contains template expressions when the author deliberately allows it. On the command line, the set syntax handles lists and nested maps awkwardly, with index brackets, braces and escaped commas and dots, so set-json is cleaner for structured input. Type issues are common: a version such as 1.20 becomes a float, a value like 0123 or yes may be reinterpreted, and large integers can become floating point and print in scientific notation. To avoid this, quote strings in values, use set-string, and apply the quote function in templates, or print integers with int64 conversions. For files embedded in config maps, the Files object and set-file load content, with the chart's files directory using glob patterns to include many at once. Testing the rendered output with edge-case inputs and schema types prevents surprises.

Likely follow-ups

- Why does 1.20 become 1.2?
- How do you embed whole files in a ConfigMap?

Use values files with native YAML and block scalars, render with toYaml and quote, use set-json or set-string on the command line, watch float and boolean coercion, and embed files with the Files object or set-file.

```
config: |
  log_level: info
  retries: 3
```

```
data:
  app.conf: |-
    {{- .Values.config | nindent 4 }}
  {{- (.Files.Glob "files/*").AsConfig | nindent 2 }}
```

Interview tip: Say quote versions; large ints go wrong.

📚 Official docs

- Helm: Values
- Helm: Charts and dependencies

certs.eknathalabs.com

Q30 How would you validate and test values and rendered output before deploying?

Values & config

🎙 Spoken answer · ~70 s

I use several layers. First, schema validation in lint and template catches wrong types and missing required values. Second, helm lint with the strict flag finds chart problems. Third, helm template renders manifests, which I pipe into kubeconform or kubeval to validate against Kubernetes API schemas for the target version, including custom resource schemas, and into policy tools like conftest with OPA or Kyverno CLI to enforce rules such as resource limits, no latest tags, required labels and non-root users. Fourth, helm install with dry-run against a real cluster validates admission and server-side checks, and the diff plugin shows changes against the live state. For templates themselves, helm-unittest runs unit tests with assertions on rendered documents for different value inputs, and chart-testing, the ct tool, lints and installs changed charts in a kind cluster with test values. Snapshots of rendered output across environments reveal unintended differences in review. All of this runs in continuous integration for every pull request, and the same rendered manifests are what gets deployed.

Likely follow-ups

- What does kubeconform validate?
- How does helm-unittest work?

Layer schema validation, helm lint --strict, rendering to kubeconform and policy checks with conftest or Kyverno, server-side dry-run and helm diff, helm-unittest assertions, and chart-testing installs in kind, all in CI.

```
helm lint ./chart --strict -f values/prod.yaml
```

```
helm template app ./chart -f values/prod.yaml | kubeconform -strict -kubernetes-version 1.29.0
```

```
helm unittest ./chart
```

Interview tip: Say lint, schema, policy, dry-run, diff, install test.

📚 Official docs

- Helm: Values

certs.eknathalabs.com

Q31 How do you provide sensible defaults for resources, replicas and autoscaling in a reusable chart?

Charts & dependencies

🎙 Spoken answer · ~75 s

Defaults should let the chart run safely for evaluation yet make production tuning obvious. For resources, I either set modest requests and limits or leave the block empty with a comment, but a published chart should avoid silently running without limits in shared clusters, so many charts ship small defaults that users override. Replicas default to one for development with guidance to raise it, and when the autoscaling option is enabled the template must omit the replicas field from the deployment, otherwise every upgrade resets the horizontal autoscaler's scaled count. A pod disruption budget option, anti-affinity or topology spread constraints, and probes are exposed so availability can be configured. Resources are rendered with toYaml so users can set any combination. I give example production values in the documentation, for instance requests based on observed usage, and I base numbers on measurement rather than guessing. Presets such as small, medium and large can simplify choices, mapped to resource blocks in a helper while still allowing explicit override. Defaults are reviewed on each major version, since changing them affects existing users.

Likely follow-ups

- Why omit replicas when autoscaling is enabled?
- What should a chart do about limits by default?

Ship safe modest defaults or documented empty blocks, render resources with toYaml, omit replicas when HPA is enabled to avoid upgrade resets, expose PDB, spread and probes, offer size presets and give measured production examples.

```
{{- if not .Values.autoscaling.enabled }}
replicas: {{ .Values.replicaCount }}
{{- end }}
```

Interview tip: Say omit replicas when the HPA owns scaling.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q32 What is the structure of a Helm chart and what does each part do?

Charts & dependencies

🎙 Spoken answer · ~71 s

A chart is a directory with a defined layout. Chart.yaml holds metadata: API version, name, chart version, application version, description, type, dependencies in version 2, keywords, maintainers and the supported Kubernetes version constraint. Values.yaml contains default configuration, and an optional values.schema.json validates it. The templates directory holds the manifest templates, helper templates in an underscore file, a notes text file displayed after install, and a tests subdirectory with test pod definitions run by the helm test command. The charts directory contains dependency subcharts, either vendored or downloaded by the dependency update command. The crds directory holds custom resource definitions that are installed before templates but never upgraded or removed by Helm. A chart lock file records exact resolved dependency versions. A helmignore file excludes files from packaging. A readme and license document the chart, and an optional files directory carries extra files that templates load. The helm create command generates this skeleton with a sample deployment, service, ingress, service account, autoscaler and helpers, which makes a good starting point after trimming.

Likely follow-ups

- What goes in the crds directory?
- What is Chart.lock for?

Chart.yaml has metadata and dependencies, values.yaml and optional schema hold configuration, templates contains manifests, helpers, notes and tests, charts holds subcharts, crds holds CRDs, Chart.lock pins dependencies and .helmignore excludes files.

```
helm create mychart && tree mychart
```

Interview tip: Say Chart.yaml, values, templates, charts, crds.

📚 Official docs

- Helm: Chart best practices
- Helm: Charts and dependencies
- Helm documentation

certs.eknathalabs.com

Q33 What is the difference between chart version and appVersion, and how should you version charts?

Charts & dependencies

🎙 Spoken answer · ~70 s

The chart version in Chart.yaml identifies the packaging, templates and defaults, and must follow semantic versioning, since Helm uses it for dependency resolution, repository indexing and upgrade comparison. The appVersion describes the version of the application being deployed, such as the container image tag, and is informational for Helm: it is shown in listings and commonly used as the default image tag in templates. They change independently. A new application release with no template changes bumps appVersion and, because the packaged chart content changed, should also bump the chart version, since repositories require a new chart version for every published change. Template or values changes bump the chart version by their impact: patch for fixes, minor for backwards-compatible features and a major bump for breaking changes such as renamed values, removed options or changed selectors. Release notes or a changelog list upgrade steps. Automating version bumps in the release pipeline, and using chart-releaser or similar to publish, avoids human error. Never overwrite a published version; publish a new one.

Likely follow-ups

- Does Helm interpret appVersion?
- When is a major chart bump needed?

Chart version is semver for the package and is used by Helm for resolution and upgrades; appVersion is informational, often the default image tag. Bump the chart version with every change, major for breaking changes, and never overwrite a published version.

```
apiVersion: v2
name: myapp
version: 2.3.0
appVersion: "1.18.4"
```

```
helm package ./myapp --version 2.3.1 --app-version 1.18.5
```

Interview tip: Say chart version is the package; appVersion is the app.

📚 Official docs

- Helm: Plugins
- Helm: Charts and dependencies
- Helm documentation

certs.eknathalabs.com

Q34 How do chart dependencies and subcharts work?

Charts & dependencies

🎙 Spoken answer · ~72 s

Dependencies are declared in Chart.yaml under the dependencies list, each with a name, version constraint, repository location and optional alias, condition, tags and import-values. Running helm dependency update resolves the constraints, downloads the packaged subcharts into the charts directory and writes Chart.lock with exact versions and a digest, while dependency build restores exactly the locked versions, which is what CI should use for reproducibility. The same chart can appear several times using aliases, each with its own values. Conditions tie a dependency to a boolean in values, for example installing an embedded database only when enabled, and tags group dependencies to toggle together. Subcharts render as part of the parent release, share the release name and namespace, and cannot reference parent values apart from globals. Drawbacks include tight coupling, version pinning work and the temptation to embed stateful databases that deserve independent lifecycle, so for production I often prefer managing such components as separate releases or operators, with the dependency used for development convenience. Vendoring subchart archives into Git offers repeatable builds.

Likely follow-ups

- Difference between dependency update and build?
- Why avoid embedding production databases as subcharts?

Declare dependencies in Chart.yaml with versions, aliases, conditions and tags; dependency update resolves and writes Chart.lock, dependency build restores it for reproducible CI. Subcharts share the release but are isolated by values, and stateful ones are often better as separate releases.

```
dependencies:
  - name: postgresql
    version: 15.x.x
    repository: oci://registry-1.docker.io/bitnamicharts
    condition: postgresql.enabled
```

```
helm dependency build ./chart
```

Interview tip: Say lock file plus dependency build for reproducibility.

📚 Official docs

- Helm: Charts and dependencies
- Helm documentation

certs.eknathalabs.com

Q35 What are library charts and when are they useful?

Charts & dependencies

🎙 Spoken answer · ~70 s

A library chart has type library in Chart.yaml. It contains only named templates and helpers and cannot be installed, because it produces no manifests of its own. Other charts declare it as a dependency and call its templates with include, so organisations can share common patterns such as standard labels, a baseline deployment, service, ingress, config map or pod security settings, implemented once. Application charts then become thin: they provide a small values file and call the library's templates, which leads to consistent behaviour, fewer copies of the same boilerplate and a single place to fix problems or roll out a policy change. Trade-offs include indirection that makes debugging harder, tight coupling to the library's version so upgrades must be coordinated, and the need for good documentation and tests of the library, since a mistake affects all consumers. Versioning follows semantic versioning with careful change management. Alternatively, a single generic chart parameterised for many applications can serve similar goals. Library charts shine in platform teams standardising many services.

Likely follow-ups

- Can a library chart be installed?
- What is the risk of a shared library?

A library chart of type library contains only named templates and is consumed as a dependency via include, centralising labels, deployments and policies for many services. Benefits are consistency and less boilerplate; risks are indirection and wide blast radius, so version and test it carefully.

```
type: library
```

```
{{ include "common.deployment" (dict "root" . "name" "web") }}
```

Interview tip: Say library charts share templates, not manifests.

📚 Official docs

- Helm: Library charts
- Helm documentation

certs.eknathalabs.com

Q36 How do Helm hooks work and what are typical uses?

Charts & dependencies

🎙 Spoken answer · ~72 s

Hooks are ordinary manifests, usually jobs or pods, with an annotation that tells Helm to run them at a specific point in the release lifecycle: pre-install, post-install, pre-upgrade, post-upgrade, pre-delete, post-delete, pre-rollback, post-rollback and the test hook. Helm creates the hook resources at that stage, waits for jobs and pods to complete before proceeding, and a failure fails the operation. The weight annotation orders several hooks of the same kind, and the delete policy annotation controls cleanup, such as deleting before the next creation, on success, or on failure. Typical uses are database schema migrations before an upgrade, seeding data after install, taking a backup before deletion, or running smoke tests. Caveats are that hook resources are not part of the release's tracked manifest, so they are not removed by uninstall unless a policy deletes them; hooks block the rollout and need timeouts; migrations should be idempotent and backward compatible, because a rollback does not undo them; and in GitOps tools hooks map to their own sync hook concepts, so behaviour can differ.

Likely follow-ups

- Which annotation controls hook cleanup?
- Why are migration hooks risky?

Hooks are annotated jobs or pods that run at lifecycle points such as pre-upgrade or post-install, ordered by weight and cleaned by delete policy. Use for migrations, backups and tests, but make them idempotent, set timeouts and remember rollbacks do not undo them.

```
metadata:
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "0"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
```

Interview tip: Say hooks run jobs at lifecycle points; clean them up.

📚 Official docs

- Helm: Chart hooks
- Helm documentation

certs.eknathalabs.com

Q37 How do you manage Custom Resource Definitions with Helm?

Charts & dependencies

🎙 **Spoken answer · ~71 s**

Helm treats CRDs specially. Files in the chart's crds directory are installed before the templates, as plain YAML without templating, and Helm skips them if they already exist. It never upgrades or deletes them, because changing or removing a CRD can destroy custom resources and data across the cluster. This leads to the well-known problem that CRD updates in a newer chart version are not applied on helm upgrade. Strategies include applying CRDs separately with kubectl or a GitOps step before upgrading the chart, publishing a second chart that contains only the CRDs with its own lifecycle, or placing CRDs in the templates directory with a keep resource policy and a flag to toggle installation, which allows templating and upgrades but loses the safety, and requires care with ordering and with the first install when resources need the CRD already present. Many projects document a manual CRD upgrade step. Whichever approach, I treat CRDs as cluster-level, version them carefully, make conversions backward compatible, and test upgrades on a staging cluster.

Likely follow-ups

- Why does helm upgrade not update CRDs in the crds folder?
- What is the separate CRD chart approach?

Files in crds install first without templating and are never upgraded or deleted by Helm. Manage updates with a separate CRD chart, a kubectl or GitOps step, or templated CRDs with a keep policy, and test upgrades because removal destroys custom resources.

```
kubectl apply --server-side -f https://example.com/crds-v2.yaml
```

```
helm upgrade --install app ./chart --skip-crds
```

Interview tip: Say Helm installs CRDs once and never touches them again.

📚 **Official docs**

- Kubernetes: Custom Resources
- Helm: Chart template guide
- Helm: CRDs

certs.eknathalabs.com

Q38 How do you write and run Helm tests?

Charts & dependencies

🎙 Spoken answer · ~72 s

Helm tests are manifests, typically pods or jobs, placed in the templates tests directory and annotated with the test hook. They are not run during install or upgrade. After a release exists, running helm test creates those pods, waits for them to finish and reports pass if each exits with zero, failing otherwise, and logs can be printed with the logs flag. A test commonly checks that the service responds, for instance a pod running curl or wget against the service's cluster address and expecting a success status, or verifying database connectivity or a login flow. Tests should be fast, read-only or self-cleaning, use minimal images, and have unique names and delete policies so reruns do not collide. They give a basic post-deploy smoke check usable in pipelines after an upgrade, with a failure triggering rollback. They are not a replacement for unit tests of templates, with helm-unittest, or end-to-end testing. The chart-testing tool can run install plus test in a throwaway kind cluster for each pull request, making chart quality checkable automatically.

Likely follow-ups

- When do Helm tests run?
- How do you clean up test pods?

Test manifests in templates/tests carry the helm.sh/hook: test annotation and run only on helm test, passing on exit code zero. Use them as smoke checks in pipelines with cleanup policies, alongside unit tests and chart-testing in kind.

```
metadata:
  annotations:
    "helm.sh/hook": test
spec:
  containers:
    - name: probe
      image: curlimages/curl
      command: ["curl", "-fsS", "http://{{ include \"mychart.fullname\" . }}:80/healthz"]
```

```
helm test app --logs
```

Interview tip: Say helm test is a post-deploy smoke test.

📚 Official docs

- Helm: Chart tests
- Helm documentation

certs.eknathalabs.com

Q39 How do you package and structure charts for many microservices: one chart per service, umbrella charts or a shared generic chart?

Charts & dependencies

🎙 **Spoken answer · ~69 s**

There are three common models. One chart per service gives each team independent versioning, release cadence and flexibility, at the cost of duplicated boilerplate unless a library chart supplies shared templates. An umbrella chart bundles many services as dependencies and deploys them together as one release, simple for a small product that always ships as a unit, but it couples versions, creates large blast radius, slows upgrades, and makes independent rollback of one service impossible. A shared generic chart, parameterised by values for each service, centralises best practice, and teams only supply values, ideal for platform teams with uniform workloads, though it can become over-configurable and awkward for unusual services. Many organisations combine a library chart for common helpers with a thin chart per service, or a generic application chart with per-service values in GitOps. I weigh team autonomy against consistency, release coupling and operational complexity. Whatever the choice, charts are versioned, published to a registry, tested in CI and documented, and rollouts happen per service.

Likely follow-ups

- Why are umbrella charts risky for independent services?
- What is the benefit of a generic chart?

One chart per service gives independence, umbrella charts couple services into one release, and a shared generic chart centralises practice. Often a library chart plus thin per-service charts, or generic chart with per-service values, balances consistency and autonomy.

```
helm upgrade --install orders oci://registry/charts/generic-app --version 3.2.0 -f
services/orders/values.yaml
```

Interview tip: Say balance autonomy against consistency.

📚 **Official docs**

- Helm: Charts and dependencies
- Helm documentation

certs.eknathalabs.com

Q40 How do you create a chart from scratch with helm create and turn the scaffold into something production-ready?

Charts & dependencies

🎙 **Spoken answer · ~71 s**

Helm create generates a working skeleton with a deployment, service, ingress, service account, horizontal autoscaler, helpers, notes and a test. It is a reasonable start, but I adapt it deliberately. I remove templates I do not need, set sensible image, resources and probe defaults, and replace the sample nginx image settings with the real application's. I check labels follow the recommended Kubernetes label set, selectors are stable, and names use the fullname helper. I add a pod disruption budget, topology spread or anti-affinity options, security context defaults such as non-root and read-only root filesystem, resource requests and limits, and configurable environment variables, volumes and annotations. Values get comments and a schema, and the readme is generated by helm-docs. Config checksums trigger rollouts on config changes, and image tags default to appVersion but allow digests. I add unit tests, a smoke test, chart-testing configuration and a lint step in CI. Finally I set chart version and appVersion, a maintainers and kubeVersion field, and publish to a registry with provenance if required.

Likely follow-ups

- Which security defaults do you add?
- Why allow image digests?

Start from helm create, trim unused files, fix names and labels, add PDB, spread, security context, resources and probes, document values with helm-docs and a schema, add checksums, unit and smoke tests, CI lint and publish with proper versions.

```
helm create webapp
```

```
helm lint webapp --strict && helm-docs
```

Interview tip: Say scaffold, harden, document, test, publish.

📚 **Official docs**

- Helm: Chart template guide
- Helm documentation

certs.eknathalabs.com

Q41 How do Helm rollbacks work, and what are their limits?

Releases & lifecycle

🎙 Spoken answer · ~74 s

Every install or upgrade creates a numbered revision stored in the cluster. Helm rollback takes a release to a specified earlier revision, or the previous one by default, by computing a patch between the current state and that revision's stored manifest and applying it, which itself creates a new revision rather than deleting history. Hooks for rollback run, and flags such as wait, timeout, cleanup-on-fail and force control behaviour. Limits matter. Rollback restores Kubernetes resources, not data, so a database migration applied by an upgrade is not reverted, and an old application version may not work with the new schema unless migrations are backward compatible. Resources that were changed manually are merged under the three-way strategy. CRDs are not rolled back. Persistent volume claims and externally created resources stay. If the history limit pruned old revisions, they are unavailable. Rollback also cannot fix a bad configuration that exists in every revision. In GitOps setups, rolling back with Helm directly creates drift from Git, so the correct path is to revert the commit and let the controller reconcile.

Likely follow-ups

- Does rollback revert database changes?
- Why revert Git in GitOps instead of using helm rollback?

Rollback applies a previous revision's manifest as a new revision, running rollback hooks. It does not revert data, migrations, CRDs or manual changes cleanly, depends on retained history, and in GitOps should be done by reverting Git.

```
helm history app -n prod
```

```
helm rollback app 7 -n prod --wait --timeout 5m
```

Interview tip: Say rollback reverts manifests, not data.

📚 Official docs

- Kubernetes: Deployments
- Helm: Chart hooks
- Helm: Rollback

certs.eknathalabs.com

Q42 What do --atomic, --wait, --timeout and --cleanup-on-fail do in an upgrade?

Releases & lifecycle

🎙 Spoken answer · ~74 s

By default, Helm returns once the Kubernetes API has accepted the resources, not when the application is healthy. The wait flag makes Helm poll until pods, deployments, stateful sets, jobs and services are ready, up to the timeout, which defaults to five minutes, so the command's exit status reflects real readiness. The atomic flag implies wait and adds automatic rollback to the previous revision if the upgrade fails or times out, or removes the release entirely in the case of a failed install, leaving the cluster in its last known good state and the pipeline red. The cleanup-on-fail flag deletes new resources that were created during a failed upgrade, preventing leftover objects. Wait for jobs makes Helm also wait for job completion. The timeout should reflect real startup time, including image pulls and migrations, and also bound hooks. These flags make pipelines safer, but atomic rollback can mask the actual error unless logs and events are captured first, and a rollback itself may fail if data changed. I capture pod status and events on failure for debugging.

Likely follow-ups

- What is the default behaviour without wait?
- What does atomic do on a failed install?

Wait polls until resources are ready within the timeout, atomic implies wait and rolls back on failure or removes a failed install, and cleanup-on-fail deletes new resources from a failed upgrade. Set realistic timeouts and capture events before rollback.

```
helm upgrade --install app ./chart --atomic --timeout 10m --cleanup-on-fail
```

Interview tip: Say atomic makes failed deploys revert themselves.

📚 Official docs

- Helm: Rollback
- Helm: Upgrade

certs.eknathalabs.com

Q43 What does a Helm release in a failed or pending state mean, and how do you recover from pending-upgrade or failed states?

Releases & lifecycle

🎙 Spoken answer · ~72 s

A release has a status such as deployed, failed, superseded, uninstalled, pending-install, pending-upgrade or pending-rollback. Pending states appear while an operation is in progress, and if the client is interrupted, for example a CI job killed mid-upgrade, the release can be stuck in pending-upgrade, after which further upgrades fail with an error saying another operation is in progress. Recovery begins with checking helm history and status, and confirming no Helm process is actually still running. A common fix is a rollback to the last deployed revision, which creates a new one and clears the stuck state. If that fails, the stuck release secret can be inspected and the last revision's status label edited or the bad revision's secret deleted with kubectl, a careful manual intervention, after which an upgrade can proceed. A failed release can simply be upgraded again with corrected values, or uninstalled and reinstalled if it is a new install. To prevent this, use atomic and timeouts, avoid cancelling pipelines abruptly, run deployments serialised with concurrency groups, and keep enough history.

Likely follow-ups

- Why does an interrupted CI job cause pending-upgrade?
- How do you manually unstick a release?

Pending states mean an operation was interrupted; check history and processes, then helm rollback to the last good revision or, as last resort, fix or delete the stuck revision secret. Prevent with atomic, timeouts and serialised pipelines.

```
helm history app -n prod
```

```
helm rollback app 12 -n prod
```

```
kubectl get secret -n prod -l owner=helm,name=app,status=pending-upgrade
```

Interview tip: Say rollback first; edit secrets last.

📚 Official docs

- Kubernetes: Debug Pods
- Helm: Chart hooks
- Helm: Upgrade

certs.eknathalabs.com

Q44 How does Helm compute upgrades with the three-way strategic merge, and what happens to manual changes made with kubectl?

Releases & lifecycle

🎙 **Spoken answer · ~74 s**

During upgrade, Helm compares three things: the manifest of the old release, the live state of the objects in the cluster, and the new manifest, and generates a patch. Fields changed between the old and new manifest are applied. Fields changed manually in the live object but not touched by the chart are preserved, which is the improvement over Helm 2's two-way comparison. But if the chart sets a field that was manually changed, the new manifest wins on upgrade, and if a field existed in the old manifest and was removed in the new, it is deleted. This means manual edits outside Helm's knowledge may persist unexpectedly, causing drift between Git and the cluster, or be overwritten without warning. Another consequence is that rollback after manual change behaves unexpectedly. Objects not part of any Helm release are not touched. Adopting existing resources requires labels and annotations for Helm ownership. The practical lesson is to avoid manual changes, use diff before upgrades and let GitOps reconcile continuously, optionally using server-side apply to manage field ownership more precisely.

Likely follow-ups

- What was wrong with the Helm 2 approach?
- How can you see drift before upgrading?

Helm patches using old manifest, live state and new manifest, so manual changes to fields the chart does not set persist while chart-set fields are overwritten and removed fields deleted. Avoid manual edits, use helm diff and GitOps, and consider server-side apply.

```
helm diff upgrade app ./chart -f values.yaml --three-way-merge
```

```
kubectl get deploy app -o yaml | diff - rendered.yaml
```

Interview tip: Say compare old, live and new.

📚 **Official docs**

- RFC 9293: TCP
- Helm: Chart template guide
- Helm: Upgrade

certs.eknathalabs.com

Q45 How do you adopt existing Kubernetes resources into a Helm release, or move a release between namespaces?

Releases & lifecycle

🎙 Spoken answer · ~68 s

Helm refuses to install resources that already exist and are not owned by the release, with an error about missing ownership metadata. To adopt a resource, add the label app.kubernetes.io/managed-by with value Helm and the annotations meta.helm.sh/release-name and meta.helm.sh/release-namespace matching the intended release, using kubectl label and annotate; then an install or upgrade that includes that resource succeeds and takes ownership. Doing this for many resources is scripted, and backups of the objects are taken first. Moving a release between namespaces is not supported directly because releases are namespaced and stored there. The safe approach is to install a new release in the target namespace, migrate data and traffic with care, then uninstall the old one, taking care that persistent volumes and external resources survive, perhaps by retaining volume reclaim policies. Renaming a release has the same constraint. Tools or manual copying of release secrets are risky and unsupported. In production, I plan migrations with maintenance windows, validate in staging and keep rollback paths.

Likely follow-ups

- Which labels mark Helm ownership?
- Why can't a release simply be moved to another namespace?

To adopt resources add the managed-by Helm label and the release-name and release-namespace annotations; to move a release, install a new one in the target namespace, migrate data and traffic, then remove the old. Releases are namespaced so direct moves are unsupported.

```
kubectl label deploy app app.kubernetes.io/managed-by=Helm
```

```
kubectl annotate deploy app meta.helm.sh/release-name=app meta.helm.sh/release-namespace=prod
```

Interview tip: Say label and annotate to adopt.

📚 Official docs

- Helm: Chart hooks
- Helm documentation

certs.eknathalabs.com

Q46 What deployment strategies can you implement with Helm, such as blue-green and canary?

Releases & lifecycle

🎙 **Spoken answer · ~72 s**

Helm itself performs standard Kubernetes updates, so rolling update behaviour comes from the deployment's strategy settings, tuned with max surge, max unavailable, readiness probes and disruption budgets. For blue-green, I install two releases, for example blue and green, each with its own deployment, and a shared service or ingress whose selector or weight switches traffic once the new release passes checks; rollback is switching back, and the old release is removed later. Another form keeps one chart with a colour value and flips the service selector. For canary, I run a second release of the new version with a small replica count behind the same service selector, giving a rough traffic share proportional to replica counts, or better, use an ingress controller's canary annotations, a service mesh or Argo Rollouts, which provide weighted traffic, metric analysis and automated promotion or abort. Argo Rollouts and Flagger integrate with Helm charts by templating a Rollout resource. Database changes need expand-and-contract migrations so both versions work. The choice depends on risk, infrastructure cost and tooling.

Likely follow-ups

- How can replica counts approximate a canary?
- What do Argo Rollouts add?

Helm does rolling updates via deployment strategy; blue-green uses two releases and switches service or ingress, canary uses a small second release or weighted ingress, mesh, Argo Rollouts or Flagger for metric-driven promotion. Use backward-compatible migrations.

```
helm upgrade --install app-green ./chart --set color=green
```

```
kubectl patch svc app -p '{"spec":{"selector":{"color":"green"}}}'
```

Interview tip: Say Helm ships; a rollout controller shifts traffic.

📚 **Official docs**

- Google SRE Workbook: Canarying Releases
- Argo Rollouts
- Martin Fowler: BlueGreenDeployment

certs.eknathalabs.com

Q47 How do you manage Helm release history and cleanup, and what is --history-max?

Releases & lifecycle

🎙 **Spoken answer · ~72 s**

Each revision is stored as a secret in the release namespace, containing the chart, values and rendered manifest. Without limits, frequently deployed releases accumulate many large secrets, which consume etcd space, slow listing and in extreme cases hit object size limits. The history-max option on upgrade, defaulting to ten, keeps only that number of recent revisions and prunes older ones, which also limits how far back a rollback can go. I choose a value that balances audit and rollback needs against storage, often five to ten, and set it consistently in pipelines. For long-term audit, Git history, pipeline logs and exported release data serve better than keeping hundreds of cluster revisions. Uninstalled releases kept with keep-history show up in listings with the all flag and should be cleaned eventually. Failed or superseded revisions are cleaned by the same pruning. I monitor etcd size and the count of Helm secrets, and avoid putting large files into charts or values, because every revision stores them again, which can make a release exceed secret size limits.

Likely follow-ups

- What is the default history limit?
- Why can large charts break releases?

Each revision is a secret, so cap them with --history-max (default 10) to protect etcd and listing speed, balancing rollback depth. Use Git and logs for audit, clean kept history and avoid large embedded files that hit secret size limits.

```
helm upgrade --install app ./chart --history-max 5
```

```
kubectl get secrets -A -l owner=helm --no-headers | wc -l
```

Interview tip: Say cap revisions; Git is the audit trail.

📚 **Official docs**

- Helm: Chart hooks
- Helm: Rollback
- Helm documentation

certs.eknathalabs.com

Q48 How do you handle database migrations alongside Helm upgrades safely?

Releases & lifecycle

🎙 Spoken answer · ~74 s

Migrations are the riskiest part of an upgrade because rollback of Kubernetes objects does not undo schema changes. The safest design is expand and contract: first release adds columns or tables in a backward-compatible way while the old application still works, the application then switches, and a later release removes unused structures. This lets old and new versions run together during rolling updates and makes rollbacks safe. Mechanically, migrations run either as a pre-upgrade hook job, which blocks the upgrade until finished and reports failure clearly but is outside release tracking and runs before new pods exist, or as an init container on each pod, which risks races, or as a separate pipeline step or operator-managed job executed once before the Helm upgrade. Migration tools take locks so only one runs at a time. Jobs need timeouts, retries off or controlled, idempotency, and resource limits. Take a backup or snapshot before risky migrations, test on a copy of production data, and monitor lock times. Never combine a destructive change with the release that stops using the data.

Likely follow-ups

- What is expand and contract?
- Why can init-container migrations be risky?

Use expand-and-contract migrations so old and new versions coexist, run them once as a pre-upgrade hook job or a dedicated pipeline step with locks, timeouts and idempotency, back up beforehand and never ship destructive changes together with the code that stops using the data.

```
metadata:
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-delete-policy": before-hook-creation
spec:
  backoffLimit: 0
```

Interview tip: Say schema changes must be backward compatible.

📚 Official docs

- Helm: Chart hooks
- Helm: Upgrade
- Helm documentation

certs.eknathalabs.com

Q49 How do you use helm diff and a dry-run to review a change before production?

Releases & lifecycle

🎙 Spoken answer · ~73 s

Before upgrading, I want to see exactly what will change. Helm template shows the full rendered manifests, but comparing to the live cluster needs the diff plugin: helm diff upgrade renders the new manifests and compares them with the release's current state, printing additions, removals and changed fields, with options to show only a number of context lines, to suppress secrets' values and to include the three-way merge so manual drift is visible. I run it in the pull request pipeline against the target cluster with read-only credentials and post the output as a comment for reviewers, so changes to values or chart versions are visible as real Kubernetes differences, including surprising side effects such as an image change, selector change or removed resource. Helm upgrade with dry-run and debug validates against the API server without persisting, catching admission errors. For GitOps tools, the equivalent is the diff or preview features. This review step catches mistakes that code review of templates and values alone cannot, because the rendered output depends on defaults and dependencies.

Likely follow-ups

- What does the diff plugin compare?
- Why hide secret values in diff output?

helm diff upgrade compares the rendered new manifests with live state, including three-way drift, and is run in pull requests with read-only credentials; dry-run adds server-side validation. Suppress secret values and review image, selector and removed resources.

```
helm diff upgrade app ./chart -f values/prod.yaml -n prod --suppress-secrets --context 3
```

```
helm upgrade app ./chart --dry-run --debug
```

Interview tip: Say review the rendered diff, not just the values.

📚 Official docs

- Helm: Plugins
- helm-diff plugin
- Helm documentation

certs.eknathalabs.com

Q50 How do you roll back safely when only part of a multi-release application is broken?

Releases & lifecycle

🎙 **Spoken answer · ~71 s**

First I identify which release introduced the problem, using monitoring, deployment annotations and each release's history, to avoid rolling back healthy components. If the services are independent and loosely coupled, I roll back only the faulty release with helm rollback to its last good revision, wait for readiness and verify metrics. If a change spans releases, such as an API contract shared by two services, rollback order matters: the consumer or producer must be reverted so compatibility holds, and backward-compatible interfaces and versioned APIs prevent hard dependencies. Feature flags allow disabling new behaviour without redeploying. In an umbrella chart all components share one release, so rolling back reverts everything, which is a reason to prefer separate releases. Data changes complicate matters, so I check migrations. For GitOps, I revert the offending commit so Git and cluster agree, and pause automated syncs of dependent apps during the incident. Afterwards, a post-incident review improves tests, adds canary stages and updates runbooks, and I record the rollback in the change log and alert history.

Likely follow-ups

- Why does an umbrella chart complicate partial rollback?
- How do feature flags help?

Identify the culprit release, roll back only that release with helm rollback, respect compatibility order between dependent services, use feature flags, check data migrations and in GitOps revert the commit. Separate releases make partial rollback possible.

```
helm rollback payments 14 -n prod --wait
```

```
helm history payments -n prod --max 5
```

Interview tip: Say separate releases allow surgical rollbacks.

📚 **Official docs**

- Helm documentation

certs.eknathalabs.com

Q51 How do Helm chart repositories work, and how do they differ from OCI registries?

Repos & OCI

🎙 **Spoken answer · ~75 s**

A classic chart repository is an HTTP server that serves an index.yaml file listing every chart version with its metadata, digest and download URL, plus the packaged chart archives. Clients add the repository, update to fetch the index into a local cache, then search and install by repository name and chart name. Any static host can serve one, such as object storage, a web server or GitHub Pages, and the index is generated with helm repo index or chart-releaser. Since Helm 3.8, OCI registry support is stable: charts are stored as OCI artifacts in the same container registries that hold images, referenced with an oci scheme URL, pushed with helm push and pulled directly without adding a repository or maintaining an index. Benefits of OCI are reuse of existing registry authentication, access control, replication, scanning and signing infrastructure, and no index file to rebuild or race on. Search is limited since there is no index. Many organisations now publish to OCI and keep HTTP repositories only for compatibility. Versions are immutable tags by convention, and digests give exact pinning.

Likely follow-ups

- What does index.yaml contain?
- Why is OCI simpler to operate?

HTTP repositories serve an index.yaml plus chart archives; OCI registries store charts as artifacts, pushed and pulled by reference with no index and reuse existing registry auth, replication and scanning. Pin by version or digest.

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
```

```
helm push mychart-1.2.0.tgz oci://registry.example.com/charts
```

Interview tip: Say OCI reuses the registry you already run.

📚 **Official docs**

- Helm: Chart best practices
- Helm: Chart repository guide
- Helm: Registries (OCI)

certs.eknathalabs.com

Q52 How do you publish a chart to an OCI registry and consume it securely?

Repos & OCI

🎙 Spoken answer · ~71 s

Publishing starts with packaging the chart with helm package, which produces a versioned archive from the Chart.yaml version. I log in to the registry with helm registry login using a token or a short-lived credential from the CI identity, then helm push uploads to a repository path, where the chart name and version become the artifact name and tag. Registries such as ECR, ACR, GAR, Harbor and GHCR support this. Consumers install by referencing the full oci URL and version, for example helm install with the oci reference and the version flag, and should pin exact versions, or digests, rather than floating ranges. Security measures include immutable tags so a version cannot be overwritten, repository permissions limiting who may push, scanning of chart contents with tools such as Trivy, Checkov or kubescape for misconfigurations, signing with cosign or Helm provenance files, and verification before install. Private registries need pull credentials in the pipeline or GitOps tool, such as a registry secret for Argo CD or Flux. Retention policies prevent unbounded growth.

Likely follow-ups

- Why pin a version or digest?
- What do immutable tags protect against?

Package, log in with a token or CI identity, helm push to an oci URL, and consume by pinned version or digest. Use immutable tags, push permissions, scanning, signing and verification, with pull credentials in GitOps tools.

```
helm package ./chart && helm push chart-1.4.0.tgz oci://ghcr.io/org/charts
```

```
helm install app oci://ghcr.io/org/charts/chart --version 1.4.0
```

Interview tip: Say immutable, pinned, signed.

📚 Official docs

- Helm: Chart repository guide
- Helm: Registries (OCI)
- Helm documentation

certs.eknathalabs.com

Q53 How do you sign and verify Helm charts, and what is provenance?

Repos & OCI

🎙 Spoken answer · ~69 s

Signing proves a chart came from a trusted publisher and was not tampered with. Helm's native provenance uses PGP: when packaging with the sign flag and a key, Helm creates a .prov file containing the chart's SHA-256 digest and a signature; consumers verify with the verify flag against a keyring before install or pull, which checks the signature and the archive's hash. Provenance for classic repositories involves publishing the .prov file alongside the chart. For OCI-based workflows, signing the artifact with Sigstore cosign, keyless using OIDC identity from CI or key-based, is increasingly common, and verification is enforced by admission controllers or GitOps tooling such as Flux, which can verify cosign signatures on OCI chart sources, or policy engines like Kyverno. Supply chain frameworks add SBOMs and attestations about how the chart was built. Key management is important: protect signing keys, rotate them, and publish public keys. Verification only has value if enforced automatically in the deployment path, not left optional, and combined with pinned digests.

Likely follow-ups

- What does a .prov file contain?
- Where is signature verification enforced?

Helm provenance signs the packaged chart with PGP into a .prov file and verifies on install; OCI charts are often signed with cosign. Protect keys, publish public keys and enforce verification in GitOps tools or admission policy with pinned digests.

```
helm package --sign --key 'ci' --keyring ~/.gnupg/secring.gpg ./chart
```

```
helm install app ./chart-1.4.0.tgz --verify --keyring pub.gpg
```

```
cosign sign ghcr.io/org/charts/chart:1.4.0
```

Interview tip: Say sign in CI, verify at deploy.

📚 Official docs

- GitHub: Artifact attestations (build provenance)
- Helm: Chart best practices
- Helm: Provenance and integrity

certs.eknathalabs.com

Q54 How would you set up an internal chart repository with ChartMuseum, Harbor or cloud registries?

Repos & OCI

🎙 Spoken answer · ~68 s

The choice depends on existing infrastructure. If a container registry already exists, using it as an OCI chart store is the simplest, with no new service: Harbor, Amazon ECR, Azure Container Registry, Google Artifact Registry and GitHub Container Registry all support Helm charts, with their access control, replication and vulnerability scanning. Harbor adds projects, robot accounts, retention and signing integration, and can also proxy upstream registries. ChartMuseum is a lightweight open-source HTTP repository server with storage backends such as S3, GCS or local disk, with an API to upload, and it generates the index automatically, a fit when classic repository semantics are needed. Static hosting with chart-releaser and GitHub Pages works for public charts. I plan authentication with single sign-on or tokens, authorisation by team, TLS, backups, high availability and monitoring if self-hosted, retention and cleanup rules, and mirroring of key third-party charts into the internal store for availability and supply-chain control. Pipelines publish charts only from the main branch after tests pass.

Likely follow-ups

- Why mirror third-party charts internally?
- When is ChartMuseum a good fit?

Prefer an existing OCI-capable registry such as Harbor, ECR, ACR, GAR or GHCR; use ChartMuseum when HTTP repository semantics are needed. Plan auth, TLS, retention, backups and mirroring of third-party charts, with publishing only from tested main-branch pipelines.

```
helm registry login harbor.example.com --username robot$ci
```

```
helm push chart-1.0.0.tgz oci://harbor.example.com/platform
```

Interview tip: Say reuse the registry you already have.

📚 Official docs

- Helm: Chart best practices
- Helm: Chart template guide
- Helm: Chart repository guide

certs.eknathalabs.com

Q55 How do you safely consume third-party charts such as Bitnami, ingress-nginx or Prometheus stacks?

Repos & OCI

🎙 **Spoken answer · ~71 s**

Third-party charts run with real privileges in your cluster, so I vet them before use. I review maintainers, activity, licensing and issue history; read the templates and default values for security settings, RBAC, CRDs and hooks; and render with helm template to inspect everything it would create. I pin an exact chart version and review the changes between versions with a diff before upgrading, instead of following the latest. Charts are mirrored into an internal registry or vendored into Git to avoid dependence on external availability and tampering, and image references are overridden to pull from an internal registry with scanned, approved images, ideally by digest. Customisation is done through values wherever possible, not by forking, with post-render hooks or Kustomize patches for the remainder. I run policy checks and scanners on rendered output, test upgrades in staging and track security advisories and deprecations, including recent changes in distribution of some popular chart catalogues. Ownership is assigned for each third-party chart, with a regular upgrade cadence so it does not rot.

Likely follow-ups

- Why mirror or vendor external charts?
- What is a post-renderer?

Vet third-party charts by reading rendered output, pin versions, review diffs between versions, mirror or vendor them, override images to approved internal digests, customise via values or post-renderers, scan and test in staging and assign an owner for regular upgrades.

```
helm template ingress ingress-nginx/ingress-nginx --version 4.11.3 -f values.yaml > rendered.yaml
```

```
helm upgrade --install ingress ./ingress-nginx-4.11.3.tgz --post-renderer ./kustomize-post.sh
```

Interview tip: Say pin, mirror, read the render, own the upgrade.

📚 **Official docs**

- Kubernetes: Ingress
- Artifact Hub
- NGINX documentation

certs.eknathalabs.com

Q56 What is the helm pull, helm show and helm search workflow for evaluating a chart? Repos & OCI

🎙 Spoken answer · ~71 s

Before installing a chart, I explore it without side effects. Helm search repo finds charts in added repositories, with the versions flag listing all versions, and helm search hub queries Artifact Hub. Helm show chart prints the Chart.yaml metadata, helm show readme the documentation, helm show values the full default values with comments, and helm show crds any CRDs, all without installing. Helm pull downloads the chart archive, and with the untar flag extracts it so I can read the templates, helpers and dependencies in an editor, and with the version flag fetches a particular one. Comparing two versions' extracted directories shows what changed between releases, which is useful for upgrade planning. Then helm template with my intended values renders everything for review, and a dry-run install against a test cluster checks admission. For OCI charts, the same show and pull commands accept the oci reference. This flow builds understanding of what will be created, which settings matter for production, and which values to override, before any resources reach a cluster.

Likely follow-ups

- How do you compare two chart versions?
- What does helm show values give you?

Search for the chart, helm show chart, readme, values and crds to read defaults, helm pull --untar to inspect templates, compare versions by diffing extracted directories, then helm template with intended values and a dry-run before installing.

```
helm search repo nginx --versions | head
```

```
helm show values bitnami/nginx --version 18.2.0 > defaults.yaml
```

```
helm pull bitnami/nginx --version 18.2.0 --untar
```

Interview tip: Say read before you install.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q57 How do you automate chart releases with chart-releaser or a CI pipeline?

Repos & OCI

🎙 Spoken answer · ~69 s

A release pipeline triggers when chart changes merge to the main branch. Steps are lint and unit tests, validation of the version bump so that a changed chart cannot reuse an existing version, packaging, publishing and tagging. For GitHub-based classic repositories, the chart-releaser action packages changed charts, creates GitHub releases with the archives attached, and updates the index.yaml on a gh-pages branch. For OCI, the pipeline runs helm package, logs into the registry with a short-lived token from the CI identity, pushes the artifact, and optionally signs it with cosign and attaches an SBOM. The chart-testing tool detects which charts changed with its list-changed command and enforces version increments, a maintainers file and lint rules. Release notes come from conventional commits or a changelog. Branch protection requires reviews, and publishing credentials exist only for the protected branch. Afterwards, a notification or automated pull request updates consuming repositories, for instance Renovate bumping chart versions in GitOps configs. Publishing is idempotent and refuses to overwrite existing versions.

Likely follow-ups

- How does chart-testing find changed charts?
- How do consumers learn about new versions?

On merge to main: lint, unit tests, version-bump check, package, publish with chart-releaser or helm push, sign and tag. Use ct list-changed, protected branches and short-lived credentials, never overwrite versions and let Renovate propose consumer bumps.

```
ct lint --target-branch main
```

```
cr package charts/app && cr upload -o org -r charts && cr index -o org -r charts --push
```

Interview tip: Say version bump enforced, publish only from main.

📚 Official docs

- Helm: Chart template guide
- Helm documentation

certs.eknathalabs.com

Q58 How do you search for and evaluate charts on Artifact Hub, and what signals indicate a trustworthy chart?

Repos & OCI

🎙 **Spoken answer · ~70 s**

Artifact Hub is a web catalogue and search service indexing charts from many repositories and other cloud-native artifact types. Each chart page shows the description, versions, readme, values, security report from image scanning, verified publisher and official badges, signed status, repository details and install instructions. When evaluating, I look for an official or verified publisher that matches the real project, signing and provenance, active maintenance with recent releases and responsive issues, clear documentation, a values schema, sensible security defaults, a permissive licence, healthy usage and a clean security scan of the referenced images. I also check the underlying source repository, how it handles breaking changes and upgrade notes, and whether it follows Kubernetes label conventions. Warning signs are abandoned repositories, unpinned latest tags, privileged containers by default, excessive RBAC, no tests, and sole maintainers with no security policy. Popularity alone is not trust. After shortlisting, I render the chart, review its templates and run policy scans before adopting. Artifact Hub is a discovery aid, not a security guarantee.

Likely follow-ups

- Does a high download count prove safety?
- What do verified publisher badges indicate?

Artifact Hub helps discovery; judge charts by verified publisher, signing, recent maintenance, documentation, schema, safe defaults, image scan results and licence, then read the rendered output and scan it. Popularity is not proof of safety.

```
helm search hub prometheus --max-col-width 80
```

```
helm show chart prometheus-community/prometheus
```

Interview tip: Say discovery is not endorsement.

📚 **Official docs**

- Helm: Provenance and integrity
- Artifact Hub
- Helm documentation

certs.eknathalabs.com

Q59 How do you work with private repositories and authentication in Helm, including in CI?

Repos & OCI

🎙 **Spoken answer · ~75 s**

For classic HTTP repositories, helm repo add accepts username and password flags, or client certificates and a CA bundle, and stores credentials in the local repositories file, so in CI I add the repository at run time using masked pipeline secrets, and I avoid caching that file in shared runners. Pass-credentials-all is needed if the chart download URLs are on a different host than the index, with attention to credential leakage. For OCI registries, helm registry login writes credentials into the registry config file, preferably using a credential helper or a short-lived token produced by workload identity, for example an OIDC token exchanged for an ECR or Artifact Registry token, avoiding long-lived passwords. In GitOps controllers, the repository credentials live as Kubernetes secrets or are fetched through cloud identity. Least privilege applies: pull-only credentials for deploy jobs, push rights only for the release pipeline. Tokens are rotated, scoped to one repository, and never printed in logs, and the Helm configuration directory is isolated per job and cleaned up afterwards. TLS verification stays on, with a custom CA provided if needed.

Likely follow-ups

- Why avoid long-lived registry passwords?
- What is a credential helper?

Add repositories with masked secrets at run time, log in to OCI with short-lived workload-identity tokens, keep pull-only credentials for deploys and push rights for the release job, isolate Helm config per job, rotate tokens and keep TLS verification enabled.

```
aws ecr get-login-password | helm registry login --username AWS --password-stdin
123456789.dkr.ecr.eu-west-1.amazonaws.com
```

```
helm repo add internal https://charts.example.com --username $U --password $P
```

Interview tip: Say short-lived tokens, least privilege.

📚 **Official docs**

- Helm: Chart template guide
- Helm: Chart repository guide
- Helm documentation

certs.eknathalabs.com

Q60 How do you keep a chart repository healthy: versioning rules, retention, deprecation and breaking changes?

Repos & OCI

🎙 Spoken answer · ~70 s

A repository is a product with consumers, so it needs rules. Versions are immutable and follow semantic versioning, enforced by CI so nobody republishes an existing version. Breaking changes, such as renamed values, changed selectors, removed options or new minimum Kubernetes versions, require a major bump, an entry in the changelog and an upgrade guide, ideally with a values migration script or deprecation warnings printed from the notes for several releases before removal. Chart-level deprecation is signalled with the deprecated field in Chart.yaml and a note pointing to the replacement. Retention policies prune very old versions carefully, keeping all versions still referenced by deployments, and storing at least the last several majors; deleting a version consumers pin breaks their builds. Quality is maintained through automated lint, tests and scanning on every change, a defined support window for Kubernetes versions, and an owner for each chart with an on-call or issue process. Metrics on downloads show who uses what, informing sunsetting. Communication through release notes and announcements prevents surprises.

Likely follow-ups

- How do you signal that a chart is deprecated?
- Why not delete old versions freely?

Enforce immutable semver versions, document breaking changes with majors, changelogs and upgrade guides, deprecate via Chart.yaml and notes, retain versions in use, automate tests and scans, define supported Kubernetes ranges and assign owners.

```
deprecated: true
kubeVersion: ">=1.27.0-0"
```

```
ct lint --check-version-increment
```

Interview tip: Say treat the repo like a product with consumers.

📚 Official docs

- Prometheus: Storage (TSDB)
- Helm: Chart best practices
- Google API Design Guide

certs.eknathalabs.com

Q61 How do you integrate Helm into a CI/CD pipeline, from lint to production deployment?

CI/CD & GitOps

🎙 **Spoken answer · ~75 s**

A typical pipeline has stages. On every pull request: helm lint with strict mode, schema validation, template rendering for each environment's values, validation with kubeconform and policy checks, unit tests with helm-unittest, security scans of manifests and images, and chart-testing install in an ephemeral kind cluster, plus a helm diff against the target environment posted as a comment. On merge: package and publish the chart to the registry with a new version, build and push the application image, and update the environment's values or chart reference. Deployment to staging runs helm upgrade --install with atomic, wait and timeout, followed by smoke tests via helm test and automated checks, and promotion to production requires approval, uses the same artefacts and a canary or progressive step if available, and verifies metrics afterwards with automatic rollback on failure. Pipeline identity uses short-lived cloud credentials with least privilege per environment, deployments are serialised with concurrency controls, and everything is logged, with notifications and a record linking commit, chart version and release revision. The exact same chart and image are promoted between environments, only values differ.

Likely follow-ups

- Why promote the same artefact between environments?
- What runs on pull requests versus merges?

PRs run lint, schema, render, kubeconform, policy, unit tests, scans, kind install tests and a diff; merges publish chart and image; deploys use upgrade --install with atomic and wait, smoke tests, approvals and metric checks, promoting identical artefacts with environment values.

```
helm lint chart --strict && helm unittest chart
```

```
helm upgrade --install app oci://reg/charts/app --version $V -f env/$ENV.yaml -n $NS --
atomic --wait --timeout 10m
```

Interview tip: Say same artefact, different values.

📚 **Official docs**

- Helm: Using Helm / commands
- Helm documentation

certs.eknathalabs.com

Q62 What is GitOps with Helm, and how do Argo CD and Flux deploy Helm charts?

CI/CD & GitOps

🎙 **Spoken answer · ~74 s**

GitOps keeps the desired state in Git, and a controller in the cluster continuously reconciles the cluster to match it, so deployments are pull-based, auditable and self-healing. For Helm, Argo CD uses an Application resource pointing to a chart in a Git path, a Helm repository or an OCI registry, with value files and parameters; it renders the chart with helm template and applies the manifests itself, so it does not create Helm release secrets, meaning helm list does not show them, and Helm hooks are mapped onto Argo CD sync hooks. Flux uses a HelmRelease custom resource with a HelmRepository or OCIRepository source; its helm-controller runs real Helm install and upgrade actions, so Helm releases and history exist in the cluster, with remediation settings for retries and rollback, drift detection, tests and dependency ordering between releases. In both, changes happen through pull requests that change values or versions, which gives review and rollback by revert. Secrets are handled with SOPS or external secret operators. Differences in hooks, rollback behaviour and visibility should be understood before choosing.

Likely follow-ups

- Does Argo CD create Helm releases?
- What does Flux's HelmRelease remediation do?

GitOps controllers reconcile Helm charts from Git or registries: Argo CD renders with helm template and applies manifests without Helm release records, while Flux's HelmRelease runs real Helm actions with retries, remediation and drift detection. Changes flow through pull requests.

```
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
spec:
  interval: 10m
  chart:
    spec:
      chart: app
      version: 1.4.x
      sourceRef: {kind: HelmRepository, name: internal}
```

```
argocd app diff app-prod
```

Interview tip: Say Argo renders and applies; Flux runs Helm.

📚 **Official docs**

- OpenGitOps principles
- Helm: Chart best practices
- Helm: Chart template guide

certs.eknathalabs.com

Q63 How do you promote a release across dev, staging and production using Helm?

CI/CD & GitOps

🎙 Spoken answer · ~71 s

Promotion moves the same immutable artefacts through environments while only configuration changes. The chart is versioned and published once, the application image is built once and referenced by tag or digest, and each environment has its own values file in version control. A change flows by pull request: bump the chart version or image tag in dev values, let automation deploy and test, then promote by copying the version into the staging values file, and later into production, each as a reviewed commit or an automated promotion job with gates. Gates include passing smoke tests, no alerts firing, a soak period, security scans and manual approvals for production. Tools like Argo CD ApplicationSets, Flux image automation, Kargo and Renovate help automate version bumps and promotions. Environment differences are kept minimal and visible, with a diff of rendered manifests between environments in review to ensure only intended differences exist. Rollback is reverting the promotion commit. Everything is recorded so you can answer what version is in which environment and when it changed.

Likely follow-ups

- What gates guard production promotion?
- How do you check environment parity?

Publish chart and image once, keep per-environment values in Git, promote by reviewed commits or automation through dev, staging and production with test, soak and approval gates, check parity with rendered diffs and roll back by reverting.

```
diff <(helm template a ./chart -f env/stg.yaml) <(helm template a ./chart -f env/prod.yaml)
```

```
yq -i '.image.tag = "1.4.2"' env/stg.yaml
```

Interview tip: Say build once, promote by config commits.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q64 How do you test Helm charts automatically in CI with chart-testing, kind and unit tests?

CI/CD & GitOps

🎙 **Spoken answer · ~70 s**

Chart testing uses several tools together. The ct tool, chart-testing, detects which charts changed relative to the target branch, lints them including version-increment and maintainer checks, and installs them in a real cluster, typically a kind cluster created in the pipeline, using one or more values files from a ci directory that cover different configurations, such as ingress on, persistence enabled and autoscaling on, then runs helm test. Install tests catch ordering, RBAC and API issues that rendering alone cannot. Unit tests with helm-unittest check rendered documents against expectations without a cluster, fast and precise, such as asserting that enabling autoscaling removes the replica field. Static validation with kubeconform across multiple Kubernetes versions, and policy tests using OPA or Kyverno, catch compatibility and compliance problems. Upgrade tests install the previous chart version and upgrade to the current one, verifying migrations and immutable-field problems. Matrix builds test against several Kubernetes versions. Results and rendered artefacts are saved, tests run in parallel, and flaky tests are fixed, not retried blindly.

Likely follow-ups

- Why test upgrades from the previous version?
- What do ci values files do in chart-testing?

Use ct for change detection, lint and install tests in kind with several ci values files, helm-unittest for fast rendering assertions, kubeconform and policy checks across Kubernetes versions, and upgrade tests from the previous chart version in a CI matrix.

```
ct lint-and-install --target-branch main --charts charts/app
```

```
helm unittest charts/app
```

Interview tip: Say render tests, install tests, then upgrade tests.

📚 **Official docs**

- Helm: Chart best practices
- Helm: Chart template guide
- chart-testing (ct)

certs.eknathalabs.com

Q65 How do you manage Helm deployments securely from CI, including credentials and least privilege?

CI/CD & GitOps

🎙 Spoken answer · ~70 s

The pipeline is a high-value target, so I limit what it can do. Authentication to the cluster uses workload identity or OIDC federation from the CI provider to obtain short-lived tokens, instead of stored kubeconfig files with static admin credentials. The Kubernetes identity is bound by RBAC to only the namespaces and verbs needed, for example managing deployments, services and config maps in one namespace, with cluster-scoped actions such as CRDs or namespaces done by a separate privileged process. Separate identities per environment prevent a development pipeline from touching production, and production deploys need protected branches, approvals and environments. Secrets come from the CI secret store or a vault with masking, never echoed, and set flags are avoided for secret values. Runners are ephemeral and isolated, with the Helm cache and config directory cleaned afterwards. Third-party actions and plugins are pinned and reviewed. Admission policies enforce standards even if the pipeline is compromised, and audit logs track who deployed what. Chart and image signatures are verified before deployment.

Likely follow-ups

- Why use OIDC federation rather than stored kubeconfigs?
- How does RBAC limit the blast radius?

Use OIDC workload identity for short-lived cluster tokens, namespace-scoped RBAC per environment, protected production approvals, masked secrets from a vault, ephemeral runners, pinned actions and plugins, admission policies and signature verification.

```
kubectl create rolebinding ci-deploy --clusterrole=edit --serviceaccount=ci:deployer -n app-prod
```

```
helm upgrade --install app ./chart -n app-prod --kube-as=system:serviceaccount:ci:deployer
```

Interview tip: Say short-lived identity, narrow RBAC.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q66 How would you handle a failed Helm deployment in a pipeline, including diagnostics and notifications?

CI/CD & GitOps

🎙 **Spoken answer · ~71 s**

The pipeline should fail clearly and collect evidence before any automatic rollback hides it. Using wait and a timeout makes Helm report failures. On failure, a step gathers diagnostics: helm status and history, the describe output and logs of non-ready pods, recent Kubernetes events for the namespace, rollout status of deployments, and the rendered manifest or diff. These are saved as pipeline artefacts or printed in the log with secrets redacted. If atomic is used, Helm rolls back automatically; otherwise a controlled rollback step runs, followed by a verification that the previous revision is healthy. The job then exits non-zero, marks the deployment as failed in the deployment tracking system, and notifies the team through chat or paging with a link, the commit, chart version, environment and error summary. Retries are limited to transient errors, not repeating a bad release. After the incident, flakiness, slow startups or insufficient timeouts feed back into pipeline tuning, and recurring failure modes become tests or policy checks. Production failures trigger the standard incident process.

Likely follow-ups

- Why collect diagnostics before rollback?
- Which failures deserve an automatic retry?

Use wait and timeouts, then on failure capture status, events, pod logs and describe output with secrets redacted, roll back and verify health, fail the job, record the deployment as failed and notify with links. Retry only transient errors and turn recurring causes into checks.

```
kubectl get events -n prod --sort-by=.lastTimestamp | tail -30
```

```
kubectl logs -n prod -l app.kubernetes.io/instance=app --tail=100 --all-containers
```

Interview tip: Say capture evidence first, then recover.

📚 **Official docs**

- Helm documentation

certs.eknathalabs.com

Q67 What are Helm post-renderers and how can Kustomize be combined with Helm in a pipeline?

CI/CD & GitOps

🎙 **Spoken answer · ~73 s**

A post-renderer is an executable that Helm pipes the fully rendered manifests through before sending them to the cluster; it reads YAML on standard input and writes modified YAML to standard output. It lets you adjust charts you cannot or should not fork, for example adding labels, annotations, security contexts, sidecars, image registry rewrites or resource patches, with tools such as kustomize, yq or custom scripts. A common pattern is a small shell script that writes the input to a file, runs kustomize build in a directory containing a kustomization with patches, and outputs the result. The flag is passed on install, upgrade and template. The benefits are keeping upstream charts unmodified and upgrades simple, while enforcing organisation-wide changes. Drawbacks are hidden transformations that make debugging harder, a dependency on the extra tool in all environments, and breakage when upstream resource names change. In GitOps, Argo CD supports Kustomize with Helm inflation and Flux has post-renderers in HelmRelease. Always include the post-renderer in tests and diffs so the final manifests are what you review.

Likely follow-ups

- What does a post-renderer receive and return?
- What are the downsides of post-rendering?

A post-renderer is an executable that receives rendered YAML on stdin and returns modified YAML, often via kustomize, letting you patch upstream charts without forking. Include it in tests and diffs, and beware of hidden transformations and breakage on upstream renames.

```
helm upgrade --install app ./chart --post-renderer ./post-render.sh
```

```
#!/bin/sh
cat > all.yaml && kustomize build .
```

Interview tip: Say patch the render, don't fork the chart.

📚 **Official docs**

- Kusto Query Language (KQL)
- Helm: Chart template guide
- Helm: Post-rendering

certs.eknathalabs.com

Q68 How do you handle multi-cluster and multi-region Helm deployments?

CI/CD & GitOps

🎙 Spoken answer · ~70 s

Scaling to many clusters needs a model that avoids hand-run commands. Configuration is layered: shared defaults, environment, region and cluster-specific values files generated or templated from an inventory, so each cluster's differences are small and explicit. For delivery, pipelines can loop over a cluster list, deploying progressively in waves, a canary region first, then others, with health gates between waves and the ability to halt. GitOps scales more naturally: Argo CD ApplicationSets or Flux with a fleet repository generate an application per cluster from a list or a cluster generator, each pulling its values, and a management cluster or each cluster's own controller reconciles it, which avoids central credentials to all clusters. Version drift between clusters is monitored. Kubernetes versions and add-on differences are handled by Capabilities checks and tested matrices. Secrets come from a regional secret store. Rollbacks are per cluster, and blast radius is limited by wave sizes and independent failure domains. Observability aggregates deployment status centrally, and documentation lists the cluster inventory and ownership.

Likely follow-ups

- What are ApplicationSets used for?
- Why deploy in waves?

Layer values per environment, region and cluster from an inventory, roll out in waves with health gates, and use GitOps fleet tooling such as Argo CD ApplicationSets or Flux to generate per-cluster apps. Monitor drift, test version matrices and limit blast radius.

```
generators:
  - clusters:
      selector:
        matchLabels: {env: prod}
```

```
helm upgrade --install app ./chart -f values/base.yaml -f values/region/eu-west-1.yaml --
kube-context eu-west-1
```

Interview tip: Say fleet generators and wave rollouts.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q69 How do you automate chart and image version updates with Renovate or Dependabot in a Helm and GitOps workflow?

CI/CD & GitOps

🎙 Spoken answer · ~73 s

Dependency bots keep deployments current without manual effort. Renovate understands Helm charts, values files, Chart.yaml dependencies, Flux HelmRelease versions, Argo CD application manifests and container image references in values, so it opens pull requests when a newer chart or image version appears in a repository or registry. I configure it with rules: group related updates, schedule noisy ones, auto-merge safe patch updates for non-critical charts after tests pass, require review for major versions and for sensitive components such as ingress controllers or operators, and pin by digest where wanted. Each pull request triggers the usual pipeline, including lint, template rendering and diff output, so the reviewer sees the real manifest changes between chart versions and any CRD or breaking notes in release notes that Renovate links. Dependabot supports a narrower set of Helm features, so Renovate is more common here. Combined with environment promotion, updates land first in development, then flow onward after soak and tests. This reduces drift from vulnerable or end-of-life versions and makes upgrades small, frequent and reversible rather than large and risky.

Likely follow-ups

- Why review rendered diffs of bot pull requests?
- Which updates should not be auto-merged?

Renovate opens pull requests for chart and image version bumps in Chart.yaml, values, Flux and Argo manifests; configure grouping, schedules and auto-merge for safe patches, require review for majors and critical components, and rely on CI rendered diffs to catch breaking changes.

```
{
 "extends": ["config:recommended"],
 "packageRules": [{"matchUpdateTypes": ["patch"], "automerge": true}]
}
```

Interview tip: Say small frequent bumps beat big risky ones.

📚 Official docs

- OpenGitOps principles
- GitHub Dependabot
- Helm: Chart template guide

certs.eknathalabs.com

Q70 How do you prevent two pipelines from deploying the same Helm release at the same time?

CI/CD & GitOps

🎙 Spoken answer · ~71 s

Concurrent Helm operations on one release can leave it in a pending state or produce confusing results, because Helm itself only detects an in-progress operation after the fact. To prevent overlaps, the pipeline serialises deployments per environment and release. CI systems provide concurrency groups or resource locks, for example a group key made from the environment and release name with cancel-in-progress disabled for deployments so a running upgrade is never killed halfway, with later runs queued, and only the newest commit actually deploying if intermediate ones are skipped. Deployments from different branches into the same environment are avoided by restricting production deploys to the protected main branch. In GitOps, the controller is the single actor reconciling each application, which removes this class of problem, and humans make changes only through Git. Additional protections include atomic and timeouts so operations finish or revert, a cluster-side lock using a lease or annotation for custom tooling, and alerts for releases stuck in pending states. Manual helm commands against production are discouraged and audited.

Likely follow-ups

- Why not cancel an in-progress deployment?
- How does GitOps avoid the problem?

Use per-environment concurrency groups or locks that queue deployments without cancelling a running upgrade, deploy production only from protected branches, rely on GitOps single-controller reconciliation, use atomic and timeouts and alert on stuck pending releases.

```
concurrency:
  group: deploy-prod-${{ inputs.release }}
  cancel-in-progress: false
```

Interview tip: Say queue deploys; never kill one midway.

📚 Official docs

- Helm: Chart hooks
- Helm documentation

certs.eknathalabs.com

Q71 What are the main security risks of Helm charts and how do you reduce them?

SECURITY & SECRETS

🎙 Spoken answer · ~72 s

Risks arise from what charts create and from where they come from. Charts can request privileged containers, host paths, host networking, broad cluster roles, wildcard RBAC, mutable image tags, secrets in plain text values, and hooks that run arbitrary jobs. A malicious or compromised chart is effectively code running in the cluster with the installer's permissions. Mitigations start with source trust: use known publishers, pin versions and digests, mirror to an internal registry, verify signatures and review changes between versions. Technical controls include rendering and scanning manifests with Checkov, Kubescape, Trivy or kube-linter, enforcing policy through admission controllers such as Pod Security Admission, Kyverno or OPA Gatekeeper so unsafe workloads are rejected regardless of the chart, using least-privilege service accounts and namespaced roles, limiting who can install charts and into which namespaces, scanning images, avoiding embedded secrets, and restricting network access with network policies. Release secrets are protected with RBAC and encryption at rest. Supply chain artefacts such as SBOMs and provenance support audits, and the chart repository itself is access-controlled and monitored.

Likely follow-ups

- Why is installing a chart equivalent to running code?
- Which admission controls block unsafe charts?

Charts can create privileged pods, broad RBAC, mutable images and plaintext secrets. Reduce risk with trusted pinned and signed sources, rendered-manifest scanning, admission policy, least-privilege installers, secret references, network policies and protected release secrets.

```
helm template app ./chart | kubescape scan -
```

```
helm template app ./chart | trivy config -
```

Interview tip: Say scan the render; enforce policy at admission.

📚 Official docs

- Helm: Chart best practices
- Helm documentation

certs.eknathalabs.com

Q72 How do you apply Pod Security Standards and secure defaults inside a chart?

SECURITY & SECRETS

🎙 Spoken answer · ~75 s

A secure chart makes the safe option the default. Pod and container security contexts are exposed in values with restrictive defaults: run as non-root with a numeric user id, disallow privilege escalation, drop all Linux capabilities and add back only what is required, use a read-only root filesystem with emptyDir mounts for writable paths, set the seccomp profile to runtime default, and avoid host namespaces and host paths. These settings align with the restricted Pod Security Standard, which clusters enforce through Pod Security Admission labels on namespaces, so a chart that does not comply is rejected at deployment; the chart should therefore be tested in a namespace labelled restricted. Resource requests and limits, a dedicated service account with automountServiceAccountToken disabled unless the API is needed, and minimal RBAC rules complete the baseline. Images use fixed tags or digests from trusted registries, with pull policies and pull secrets configurable. Network policies can be shipped as an optional template. Documentation lists which settings can be relaxed and why, and tests assert the defaults with helm-unittest or policy checks so regressions are caught.

Likely follow-ups

- Which capabilities should be dropped by default?
- How does Pod Security Admission enforce standards?

Default to non-root, no privilege escalation, dropped capabilities, read-only root filesystem, seccomp runtime default, no host access, resource limits and minimal RBAC with token automount off; test against restricted namespaces and assert defaults with unit tests.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: {drop: ["ALL"]}
  seccompProfile: {type: RuntimeDefault}
```

```
kubectl label ns app pod-security.kubernetes.io/enforce=restricted
```

Interview tip: Say the chart's defaults should pass restricted.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q73 How is RBAC used by Helm itself, and what permissions does a deployer need?

SECURITY & SECRETS

🎙 Spoken answer · ~72 s

Helm 3 acts with the permissions of the kubeconfig identity, so Kubernetes RBAC decides what it can do. A deployer needs rights to create, update and delete every kind of resource in the chart within the target namespace, to read and write the secrets that store release records, to list namespaces if using certain commands, and to create hook resources such as jobs. Cluster-scoped resources in a chart, such as cluster roles, cluster role bindings, CRDs, namespaces and webhook configurations, require cluster-level permissions, which are far more sensitive. Good practice is to give routine application pipelines a namespace-scoped role covering only the needed kinds, and perform cluster-level installations such as operators and CRDs through a separate, more tightly controlled pipeline or platform team. Avoid cluster-admin for CI. Since a chart can create role bindings, allowing a deployer to create roles lets them escalate unless Kubernetes escalation checks block it, so grants of rbac permissions must be limited. Audit logs show Helm's API calls. Separate identities per environment limit the blast radius of compromise.

Likely follow-ups

- Why can creating roles allow privilege escalation?
- What is split between app and platform pipelines?

Helm uses the caller's RBAC: namespaced deployers need create, update and delete on the chart's kinds plus release secrets, while cluster-scoped resources need elevated rights. Give app pipelines namespace roles, separate platform pipelines for CRDs and operators, avoid cluster-admin and limit RBAC-creating rights.

```
kubectl auth can-i create deployments -n app-prod --as=system:serviceaccount:ci:deployer
```

```
kubectl auth can-i create clusterroles --as=system:serviceaccount:ci:deployer
```

Interview tip: Say Helm has no powers of its own; RBAC rules.

📚 Official docs

- Kubernetes: RBAC
- Helm documentation

certs.eknathalabs.com

Q74 How do you scan Helm charts and rendered manifests for vulnerabilities and misconfigurations?

SECURITY & SECRETS

🎙 Spoken answer · ~69 s

Scanning covers three layers. First, chart and manifest misconfiguration: render the chart per environment with helm template and feed the YAML to tools such as Checkov, Kubescape, Trivy config, kube-linter, Polaris or KICS, which check against benchmarks for privileged settings, missing limits, host access, insecure RBAC and latest tags. Second, policy as code with conftest and OPA or Kyverno CLI enforces organisation-specific rules on the same output. Third, images referenced by the chart are extracted from the rendered manifests and scanned for vulnerabilities and secrets with Trivy, Grype or registry scanners, and SBOMs are generated. These run in pull requests, failing on defined severity thresholds, with documented exceptions that expire. Scanning must use realistic production values, because defaults may hide problems that appear when options are enabled. Results are also checked again in the registry and at runtime, since new vulnerabilities appear after release, and admission controllers provide the final gate in the cluster. Findings are triaged by owners, tracked to closure, and not simply suppressed.

Likely follow-ups

- Why scan with production values?
- Why scan again after release?

Render charts per environment and run Checkov, Kubescape, Trivy or kube-linter plus conftest or Kyverno policies, extract and scan images and generate SBOMs, fail PRs on severity thresholds with expiring exceptions, and re-scan registries and enforce at admission.

```
helm template app ./chart -f values/prod.yaml > out.yaml && checkov -f out.yaml
```

```
grep -E 'image:' out.yaml | awk '{print $2}' | sort -u | xargs -n1 trivy image
```

Interview tip: Say scan the render with real values.

📚 Official docs

- Helm: Chart best practices
- Helm documentation

certs.eknathalabs.com

Q75 How do you secure the Helm release secrets and the data they contain?

SECURITY & SECRETS

🎙 Spoken answer · ~70 s

Release records are Kubernetes secrets containing the chart, the merged values and the rendered manifests, so anyone who can read them can see configuration and any secret values passed through Helm, even though they are base64 and compressed rather than encrypted. First, restrict RBAC so only the deployment identity and administrators can get or list secrets in application namespaces, and avoid granting wide secret read access to developers. Second, enable encryption at rest for secrets in etcd, ideally with a key management service, and protect etcd backups. Third, minimise sensitive content: do not pass real secret values through Helm values, reference externally managed secrets instead, so the release record contains only names. Fourth, limit history with history-max, so old sensitive revisions are not kept indefinitely. Audit logging records access to these secrets, and alerts can fire on unusual reads. Where a different storage backend is configured through the Helm driver setting, such as config maps or SQL, equivalent protections apply. Rotate any credential that has been exposed through values.

Likely follow-ups

- Are release secrets encrypted?
- How do you keep secret values out of release records?

Release secrets hold values and manifests in base64, not encryption, so restrict RBAC, enable etcd encryption at rest, keep real secret values out of Helm by referencing external secrets, cap history, audit access and rotate anything exposed.

```
kubectl auth can-i get secrets -n app-prod --as=dev-user
```

```
helm get values app -n prod
```

Interview tip: Say anyone who reads release secrets reads your values.

📚 Official docs

- Helm: Chart hooks
- Helm: Plugins
- External Secrets Operator

certs.eknathalabs.com

Q76 How do you use SOPS, helm-secrets and External Secrets together with Helm?

SECURITY & SECRETS

🎙 Spoken answer · ~75 s

They solve different parts of the problem. SOPS encrypts structured files such as YAML values while leaving keys readable, using KMS, age or PGP keys, so encrypted secrets can live in Git with reviewable diffs of structure. The helm-secrets plugin decrypts these files on the fly during helm commands, so a pipeline with access to the key can deploy; Flux and Argo CD also have SOPS integrations. The approach is simple but the decrypted values still end up in the Helm release record in the cluster. External Secrets Operator takes a different route: the chart ships an ExternalSecret resource that names a remote secret in Vault, AWS Secrets Manager or similar, and the operator creates the Kubernetes secret in the cluster, so no secret value passes through Helm, Git or the pipeline. Workloads mount the generated secret by name. The Secrets Store CSI driver mounts secrets as volumes without creating Kubernetes secrets. I prefer external secrets for runtime credentials and use SOPS for configuration that must live in Git. Key access is restricted, rotated and audited in either design.

Likely follow-ups

- What remains exposed when using SOPS with Helm?
- Why does External Secrets keep values out of release records?

SOPS encrypts values files for Git and helm-secrets decrypts at deploy time, but decrypted values still land in release records. External Secrets or CSI drivers sync secrets from a vault so charts reference only names; prefer them for runtime credentials.

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
spec:
  secretStoreRef: {name: vault, kind: ClusterSecretStore}
  target: {name: db-credentials}
  dataFrom:
    - extract: {key: prod/db}
```

Interview tip: Say SOPS for Git, External Secrets for runtime.

📚 Official docs

- External Secrets Operator
- Mozilla SOPS
- Helm: Chart template guide

certs.eknathalabs.com

Q77 How do you enforce policy on Helm deployments with Kyverno, OPA Gatekeeper or admission webhooks?

SECURITY & SECRETS

🎙 **Spoken answer · ~72 s**

Admission policy provides a cluster-side guarantee that does not depend on every chart author or pipeline behaving well. Kyverno policies are written in YAML and can validate, mutate and generate resources; Gatekeeper uses constraint templates in Rego and constraints that apply them. Typical rules require images from approved registries and not use the latest tag, ban privileged containers and host paths, require resource limits and labels such as owner, require non-root users, restrict ingress hostnames, forbid cluster-wide roles from application namespaces and verify image signatures. Because Helm sends ordinary Kubernetes objects, these policies apply regardless of how they were produced. Rollout is gradual: start in audit mode to see violations, fix charts, then switch to enforce, with exceptions managed explicitly and time-limited. The same policies are run in CI against rendered manifests using the Kyverno CLI or conftest, so developers get feedback early, before the cluster rejects a deployment. Failures from admission show up as helm errors, so charts and documentation explain how to comply. Policies are version-controlled and tested like code.

Likely follow-ups

- Why start policies in audit mode?
- How can the same policy run in CI?

Kyverno or Gatekeeper admission policies enforce registries, no latest tags, non-root, limits, labels and signatures on whatever Helm applies. Roll out audit then enforce with expiring exceptions, and run the same policies in CI on rendered manifests.

```
kyverno apply policies/ --resource rendered.yaml
```

```
kubectl get policyreport -A
```

Interview tip: Say audit first, enforce next, test in CI.

📚 **Official docs**

- Kubernetes: Admission controllers
- OPA Gatekeeper
- Helm: Chart template guide

certs.eknathalabs.com

Q78 How do you secure the supply chain for Helm charts and the images they deploy?

SECURITY & SECRETS

🎙 Spoken answer · ~72 s

The goal is to know what runs and to be able to prove it came from a trusted build. For charts: build only in protected pipelines, publish immutable versions to an access-controlled registry, sign them with cosign or Helm provenance, and attach SBOMs and build attestations such as SLSA provenance. For images: build reproducibly from minimal base images, scan for vulnerabilities, generate SBOMs, sign them, and reference them by digest in values, since tags can move. Deployment verifies before running: GitOps controllers or admission policies check chart and image signatures against trusted identities and fail closed. Upstream charts and images are mirrored to an internal registry after review so a compromised public source cannot reach production directly, and renovation of versions flows through pull requests with diffs. Credentials for publishing are short-lived and held by pipeline identities only. Provenance data is retained for audits and incident response, for example to find every cluster running an affected version when a vulnerability is announced. Regular reviews check the pipeline's own dependencies, actions and plugins.

Likely follow-ups

- Why reference images by digest?
- What does admission-time signature verification add?

Build in protected pipelines, publish immutable signed charts and images with SBOMs and attestations, pin by digest, mirror upstream content, verify signatures at deployment and admission with fail-closed policy, and keep provenance for incident response.

```
cosign sign --yes registry/app@sha256:abc123
```

```
cosign verify registry/app@sha256:abc123 --certificate-identity-regexp
'https://github.com/org/.*' --certificate-oidc-issuer
https://token.actions.githubusercontent.com
```

Interview tip: Say sign, attest, pin by digest, verify at admission.

📚 Official docs

- Helm: Chart best practices
- Helm documentation

certs.eknathalabs.com

Q79 What should you check before installing a chart that creates ClusterRoles, webhooks or CRDs?

SECURITY & SECRETS

🎙 Spoken answer · ~74 s

Cluster-scoped resources can affect every namespace, so they need a deeper review. For roles and bindings, I read exactly which verbs, resources and namespaces they grant, looking for wildcards, secrets access, pod exec, node access, escalate or bind permissions, and who the subjects are. For validating and mutating webhooks, I check the failure policy, because a webhook set to fail closed with an unavailable backend can block all deployments in matched namespaces, including system ones; the namespace and object selectors; the timeout; and the service it calls, with its availability and TLS certificate handling. For CRDs, I verify they do not conflict with an existing group and kind, how conversions and versions work, and who owns upgrades since Helm will not update them. I render the chart and diff against what exists, check hooks and jobs it runs with elevated rights, and read the upgrade notes. Installation is first tested in a non-production cluster, with a rollback and an emergency procedure to delete webhooks if they lock up the API. Ownership and monitoring for the component are assigned.

Likely follow-ups

- What is the risk of a fail-closed webhook?
- Who owns CRD upgrades?

Review ClusterRoles for wildcards, secrets and escalate or bind rights; webhooks for failurePolicy, selectors, timeouts and backend availability; CRDs for conflicts, versions and upgrade ownership. Render and diff, test in a non-production cluster and prepare emergency removal steps.

```
helm template x ./chart | yq 'select(.kind=="ClusterRole" or
.kind=="ValidatingWebhookConfiguration")'
```

```
kubectl get validatingwebhookconfigurations
```

Interview tip: Say cluster-scoped means cluster-wide blast radius.

📚 Official docs

- Kubernetes: RBAC
- Kubernetes: Custom Resources
- Helm: Hooks

certs.eknathalabs.com

Q80 How do you troubleshoot a Helm install or upgrade that fails with 'cannot re-use a name that is still in use' or 'resource already exists'?

Troubleshooting

🎙 **Spoken answer · ~79 s**

These errors point to ownership or leftover state. For a name still in use, a release with that name exists in the namespace, perhaps in a failed or uninstalled-with-history state. Helm list with the all flag shows every state, and helm status or history show details. Options are to upgrade with install, to uninstall the failed release, or to choose another name. For a resource already exists error, the object exists in the cluster but Helm does not own it, because it was created manually, belongs to another release, or is left from an earlier failed install or an uninstall that kept resources. The message names the kind and object. Check its labels and annotations for managed-by and release-name; if it belongs to another release, naming conflicts must be fixed, for example with fullname overrides. If it is an orphan to be adopted, add the Helm ownership label and annotations, or delete it if safe and then retry. Cluster-scoped objects such as cluster roles, shared across namespaces, commonly conflict when the same chart is installed twice with the same release name, so names should include the release or namespace.

Likely follow-ups

- How do you list failed or uninstalled releases?
- How do you adopt an orphaned resource?

Name in use means a release record exists, so check helm list --all, then upgrade --install, uninstall or rename. Resource already exists means an unowned or other-release object, so inspect its Helm labels and annotations, adopt it, delete it safely or fix naming, especially for cluster-scoped objects.

```
helm list -A --all | grep app
```

```
kubectl get clusterrole app -o jsonpath='{.metadata.annotations}'
```

Interview tip: Say find the owner before deleting anything.

📚 **Official docs**

- Helm: Chart template guide
- Helm: Upgrade
- Helm: Using Helm / commands

certs.eknathalabs.com

Q81 Helm upgrade fails with 'another operation (install/upgrade/rollback) is in progress'. What do you do?

Troubleshooting

🎙 **Spoken answer · ~74 s**

This means the latest release revision is in a pending state: pending-install, pending-upgrade or pending-rollback. Normally that is transient while an operation runs, but if a pipeline was cancelled, a client crashed or a timeout killed the process, the state is stuck and Helm refuses new operations to protect consistency. First, confirm that no helm process is truly still running, by checking pipelines and jobs. Then look at helm history to see the stuck revision and the last deployed one. The usual fix is helm rollback to the last good revision, which creates a new revision and clears the pending status. If rollback is blocked, inspect the release secrets with the owner and status labels; for a stuck first install, uninstall the release and reinstall. As a last resort, delete the stuck revision's secret so that the previous revision becomes the latest, or patch its status label, with a backup of the secret first, then verify with helm status. Afterwards, find out why it was interrupted, and prevent a repeat with atomic, adequate timeouts and non-cancelling deploy pipelines.

Likely follow-ups

- How do you tell if the operation is really still running?
- What is the last-resort fix?

A stuck pending revision blocks new operations. Confirm nothing is running, view history, then helm rollback to the last good revision; for first installs uninstall and retry; as last resort back up and delete the stuck revision secret. Prevent with atomic, timeouts and non-cancelling pipelines.

```
helm history app -n prod
```

```
helm rollback app <last-deployed> -n prod
```

```
kubectl get secret -n prod -l name=app,status=pending-upgrade -o name
```

Interview tip: Say rollback to clear the pending state.

📚 **Official docs**

- Kubernetes: Deployments
- Helm: Chart hooks
- Helm: Chart template guide

certs.eknathalabs.com

Q82 A Helm upgrade fails with 'UPGRADE FAILED: ... field is immutable'. How do you handle it?

Troubleshooting

🎙 **Spoken answer · ~80 s**

Kubernetes forbids changing certain fields after creation, and Helm is asking the API server to apply a patch that touches one. Common examples are a deployment's selector, a stateful set's volume claim templates and service name, a job's template, a service's cluster IP and a persistent volume claim's storage class. It often happens after a chart upgrade changed labels used in selectors, renamed a resource, or changed the job spec. The error names the resource and field. Options depend on the case. For jobs, delete the old job, or use hook delete policies or time-to-live, so a fresh one is created. For deployment selectors or stateful set templates, the object must be deleted and recreated; for a deployment that is a short outage unless a blue-green approach is used, and for stateful sets, deleting with cascade set to orphan keeps pods and volumes while the controller object is replaced. For a PVC, a new volume and data migration may be required. Avoid the issue by keeping selectors and names stable between chart versions, treating label changes as breaking, testing upgrades from the previous version in CI, and noting such steps in upgrade guides.

Likely follow-ups

- Which fields are immutable on a deployment?
- How do you recreate a StatefulSet without deleting its pods?

Immutable field errors come from changing selectors, volume claim templates, job specs or service IPs. Delete and recreate the object, orphan-delete stateful sets to keep pods and data, migrate PVCs, and prevent recurrence by keeping selectors stable and testing upgrades from the previous chart version.

```
kubectl delete statefulset db -n prod --cascade=orphan
```

```
helm upgrade db ./chart -n prod
```

Interview tip: Say selectors are forever; treat changing them as breaking.

📚 **Official docs**

- Helm: Chart hooks
- Helm: Chart template guide
- Helm: Upgrade

certs.eknathalabs.com

Q83 Helm reports 'Error: INSTALLATION FAILED: timed out waiting for the condition'. How do you find the real cause?

Troubleshooting

🎙 **Spoken answer · ~73 s**

The message only says that readiness was not reached within the timeout when wait was used; the cause lies in the cluster. I look at the objects the release created: kubectl get pods in the namespace shows states such as Pending, ImagePullBackOff, CrashLoopBackOff, ContainerCreating or not ready. For Pending, describe the pod to see scheduling messages: insufficient CPU or memory, unbound persistent volume claims, taints or node selectors that match nothing. For image pull problems, check the image name, tag, registry credentials and pull secrets. For crash loops, read container logs, including previous logs, and probe failures in events, as misconfigured liveness or readiness probes, missing configuration or unreachable dependencies are common. For jobs and hooks, check job pods and backoff. Check events sorted by time across the namespace, and quota or admission webhook rejections. Slow but healthy startups may simply need a longer timeout. Services without endpoints or ingresses pending an external load balancer can also hold up wait. After identifying the cause, fix values or infrastructure, and rerun, using atomic so failures clean up.

Likely follow-ups

- Which pod states indicate scheduling issues?
- What if the app is healthy but slow to start?

Timeout means readiness never arrived. Inspect pods, describe and events for scheduling, PVC, image pull, probe and crash issues, read current and previous logs, check hooks and quotas, increase the timeout only for slow healthy starts, then retry with atomic.

```
kubectl get pods,pvc,events -n prod --sort-by=.lastTimestamp
```

```
kubectl describe pod -n prod -l app.kubernetes.io/instance=app
```

```
kubectl logs -n prod <pod> --previous
```

Interview tip: Say Helm only reports; pods and events explain.

📚 **Official docs**

- Helm: Upgrade
- Helm: Using Helm / commands
- Helm documentation

certs.eknathalabs.com

Q84 How do you diagnose 'YAML parse error', 'nil pointer evaluating interface' and 'error converting YAML to JSON' during rendering?

Troubleshooting

🎙 **Spoken answer · ~78 s**

These are template problems and are best fixed locally with helm template and the debug flag, which prints the failing line and the partial output. A YAML parse or converting YAML to JSON error means the rendered text is invalid YAML, typically from wrong indentation, missing nindent after toYaml, a stray dash that removed a newline, an unquoted string with special characters such as a colon or a leading asterisk, a tab character, or duplicated keys. I open the rendered file, find the reported line number in the rendered output, and trace back to the template. A nil pointer evaluating error means the template accessed a key under something that is nil, for instance reading a nested value when the parent map is absent from values, or referencing a variable out of scope after a range or with changed the dot. Fixes are to add defaults in values.yaml, guard with if, use dig or default, and use the dollar sign for root access. Adding a values schema and unit tests catches many early. Use show-only on the failing template, and compare with a minimal values file to bisect.

Likely follow-ups

- How do you find the failing line in rendered output?
- What causes a nil pointer in nested values?

Render with helm template --debug and --show-only. YAML errors usually mean bad indentation, missing nindent, unquoted special strings or tabs; nil pointers come from missing parent maps or rebound dot. Add defaults, guards, dig, $ for root, a schema and unit tests.

```
helm template app ./chart --debug --show-only templates/deployment.yaml 2>&1 | head -60
```

```
helm lint ./chart --strict
```

Interview tip: Say read the rendered output around the reported line.

📚 **Official docs**

- Helm: Chart template guide
- Helm: Using Helm / commands

certs.eknathalabs.com

Q85 How do you handle Helm errors caused by removed or deprecated Kubernetes APIs, such as 'no matches for kind ... in version'?

Troubleshooting

🎙 Spoken answer · ~75 s

When Kubernetes removes an API version, charts that still emit it fail on new clusters with a message that no matches exist for the kind in that version. Releases that were installed earlier can also fail on upgrade, because Helm's stored manifest references the removed version, even if the new chart uses a supported one. Prevention starts with scanning manifests for deprecated APIs using tools like pluto, kubent or kubeconform with the target version in CI, before cluster upgrades, and reading the Kubernetes deprecation guide. Fixing a chart means updating apiVersion and any changed fields, such as the Ingress backend format, with Capabilities checks if multiple versions must be supported. For existing releases blocked by stored removed APIs, the mapkubeapis plugin rewrites the release metadata to supported versions so upgrade can proceed. If the CRD itself is missing, the chart or operator providing it must be installed first, in the right order, or the CRD installation step fixed. After cluster upgrades, verify all releases with helm list and a dry-run upgrade, and keep charts within a supported range of versions.

Likely follow-ups

- What does mapkubeapis do?
- How do you find deprecated APIs before a cluster upgrade?

Scan with pluto, kubent or kubeconform before upgrading clusters, update chart apiVersions and fields with Capabilities checks if needed, use mapkubeapis to fix stored release metadata, ensure CRDs exist first and verify releases with dry-run upgrades afterwards.

```
pluto detect-helm -o wide
```

```
helm mapkubeapis app -n prod --dry-run
```

Interview tip: Say scan for deprecated APIs before cluster upgrades.

📚 Official docs

- Helm: Kubernetes API deprecations
- chart-testing (ct)
- Helm documentation

certs.eknathalabs.com

Q86 A release shows 'deployed' but the application is not working. How do you investigate?

Troubleshooting

🎙 **Spoken answer · ~79 s**

Helm status deployed only means the manifests were accepted and, with wait, resources became ready; it says nothing about application correctness. I verify from the outside in. Check the rollout: pods running and ready, restarts, recent events, and logs for errors. Compare what was deployed with what was intended: helm get values and manifest show the effective configuration, which often reveals a wrong value, missing override, wrong image tag, or an environment-variable typo. Check the dependencies the app needs: config maps and secrets mounted correctly with expected keys, service accounts and permissions, database connectivity, DNS and network policies. Test networking path by path: the service has endpoints, selectors match pod labels, ports and target ports line up, the ingress rules, hostnames, TLS and class are right, and load balancer health checks pass. Compare with a working environment using diff of rendered manifests or helm get values. Check probes that pass while the app is actually broken. If the problem began with an upgrade, helm diff between revisions and rollback to confirm the cause. Then fix the chart or values and add a test or alert that would have caught it.

Likely follow-ups

- Which Helm command shows effective configuration?
- How do you check service-to-pod wiring?

Deployed does not mean working. Check pods, events and logs, compare helm get values and manifest with intent, verify config, secrets, permissions, service endpoints and selectors, ingress and network policies, diff against a healthy environment or previous revision and roll back to confirm.

```
helm get values app -n prod --all && helm get manifest app -n prod | less
```

```
kubectl get endpoints app -n prod
```

```
helm diff revision app 11 12 -n prod
```

Interview tip: Say verify the render, then the wiring.

📚 **Official docs**

- Helm documentation

certs.eknathalabs.com

Q87 How do you debug Helm hooks that hang, fail or leave resources behind? Troubleshooting

🎙 Spoken answer · ~78 s

Hooks are separate resources created at lifecycle points, so troubleshooting follows the job. If an install or upgrade hangs, list jobs and pods in the namespace while it runs: a pre-upgrade job waiting to schedule, failing repeatedly or sleeping on a database lock will block Helm until the timeout. Read the job pod logs and events for the cause, such as a failing migration, an unreachable database, missing secrets that the main release creates only later, or hooks ordered before resources they depend on, since pre-hooks run before the release's resources exist. Check hook weights for order and the delete policies: without before-hook-creation, a leftover job from a previous run causes an already exists error; with hook-failed deletion you lose the logs you need, so for debugging remove that policy temporarily. Because hook resources are not tracked in the release manifest, uninstall may leave them, and they can be cleaned manually. Backoff limit and active deadline seconds should be set so a failing hook ends promptly. Use helm get hooks to see what is defined, and consider moving long migrations out of hooks into pipeline steps.

Likely follow-ups

- Why does a pre-install hook fail on missing secrets?
- What does before-hook-creation do?

Watch hook jobs and pods during the operation, read logs and events, check ordering and dependencies since pre-hooks run before release resources exist, tune weights, delete policies, backoff limit and deadlines, inspect with helm get hooks and clean leftovers manually.

```
helm get hooks app -n prod
```

```
kubectl get jobs,pods -n prod -w
```

```
kubectl logs job/app-migrate -n prod
```

Interview tip: Say pre-hooks run before your resources exist.

📚 Official docs

- Helm: Chart hooks
- Helm documentation

certs.eknathalabs.com

Q88 How do you restrict which charts, registries and namespaces teams can deploy to in a shared cluster?

SECURITY & SECRETS

🎙 Spoken answer · ~71 s

In a shared cluster, I control deployment at several layers. Identity and RBAC give each team a namespace-scoped role, so a team's pipeline can only create resources in its own namespaces and cannot create cluster roles, bindings, CRDs or webhooks. Namespaces carry resource quotas, limit ranges, network policies and Pod Security Admission labels set by the platform team. Admission policies from Kyverno or Gatekeeper restrict image registries to approved internal ones, require signed images, and can limit deployments to charts or images from approved sources by validating labels and annotations. In GitOps, the controller's project or tenant configuration limits which repositories, chart sources, destinations and resource kinds each team may use; Argo CD AppProjects and Flux multi-tenancy with service account impersonation are examples. An internal registry mirrors reviewed third-party charts, and egress rules block pulling from arbitrary public registries. Platform-level components that need cluster scope are installed by the platform team. Audit logs and reports show who deployed what and where, and exceptions go through a documented request process with expiry.

Likely follow-ups

- How does Argo CD AppProject limit a team?
- Why mirror charts internally?

Use namespace-scoped RBAC, quotas, network policies and Pod Security labels, admission policy limiting registries and requiring signatures, GitOps tenant constraints like AppProjects or Flux impersonation, internal chart mirrors and audited exceptions.

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
spec:
  sourceRepos: ["https://git.example.com/team-a/*"]
  destinations: [{namespace: team-a-*, server: "*"}]
  clusterResourceWhitelist: []
```

Interview tip: Say tenants get namespaces, not cluster scope.

📚 Official docs

- Helm documentation

certs.eknathalabs.com

Q89 A Helm upgrade changed or deleted resources unexpectedly. How do you find out why and prevent it?

Troubleshooting

🎙 **Spoken answer · ~76 s**

Surprises usually come from rendering differences or ownership. First, compare revisions with helm diff revision or by fetching manifests of the two revisions with helm get manifest and diffing them, which shows exactly what the chart rendered differently. Then explain the difference: a changed default in a new chart version, a values file not included in the command so defaults replaced earlier overrides, use of reuse-values versus reset-values, a conditional that flipped because of a renamed key, or a resource whose name changed so the old one was deleted and a new one created. Helm deletes resources that were in the previous manifest but are absent in the new one, which is expected but alarming for stateful items such as PVCs, secrets or load balancers. Manual cluster edits can also be overwritten. To prevent surprises, always review the diff before production, keep full values in Git, pin chart versions, use the resource policy keep annotation for resources that must never be deleted, avoid renaming without migration, and run policy checks that block removal of stateful resources. Backups protect against mistakes that still happen.

Likely follow-ups

- Which annotation stops Helm deleting a resource?
- How do you compare two revisions?

Diff revisions with helm diff revision or helm get manifest to see rendering changes, then trace causes like changed defaults, missing values files, reuse-values, renamed keys or resource renames. Prevent with pre-upgrade diffs, full values in Git, pinned versions and the keep resource policy.

```
helm diff revision app 11 12 -n prod
```

```
metadata:
  annotations:
    "helm.sh/resource-policy": keep
```

Interview tip: Say always diff before prod and protect stateful resources.

📚 **Official docs**

- Helm: Chart hooks
- Helm: Upgrade
- Helm documentation

certs.eknathalabs.com

Q90 How do you troubleshoot Helm dependency errors, such as 'found in Chart.yaml, but missing in charts/ directory' or lock file mismatches?

Troubleshooting

🎙 Spoken answer · ~75 s

These errors mean the declared dependencies and the contents of the charts directory disagree. The message about a dependency missing from the charts directory appears when Chart.yaml lists a dependency but the archive was never downloaded, for example after a fresh clone where charts is git-ignored. Running helm dependency build restores exactly what Chart.lock says, while dependency update re-resolves version constraints and rewrites the lock. A mismatch between Chart.lock and Chart.yaml, such as a changed version range or repository, requires a dependency update and committing the new lock. Repository problems cause failures too: the repository was not added locally, its alias differs, authentication is missing, the index is stale, or the OCI registry needs login; adding the repository and running repo update usually fixes it. Version constraints that match nothing, or a chart renamed upstream, produce not found errors. In CI, I run dependency build before lint, package and template, cache the charts or vendor them, and fail when the lock is out of date. Conditions and aliases need checking if a subchart seems to be missing from the render.

Likely follow-ups

- Difference between dependency build and update?
- Why commit Chart.lock?

Missing or mismatched dependencies need helm dependency build to restore the locked set or dependency update to re-resolve and rewrite Chart.lock. Check repositories, auth and constraints, commit the lock, and run dependency build before lint, package and template in CI.

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
```

```
helm dependency build ./chart
```

Interview tip: Say commit the lock; build in CI.

📚 Official docs

- Terraform: Dependency lock file
- Helm: Charts and dependencies
- Helm documentation

certs.eknathalabs.com

Q91 How do you use the lookup function, and what are its limitations?

Hooks/tests/advanced

🎙 Spoken answer · ~75 s

Lookup queries the live cluster from within a template: it takes an API version, kind, namespace and name, and returns the object as a map, or a list when the name is empty, or an empty result if nothing is found. Typical uses are reusing an existing secret value instead of regenerating a random password on every upgrade, checking whether a resource or CRD exists before creating something, or reading a config value from another object. The limitations are important. During helm template and dry-run client mode there is no cluster connection, so lookup returns empty, meaning the same chart renders differently offline than online, and the diff plugin and GitOps renderers such as Argo CD, which run helm template without access, may produce unexpected results, for example regenerating secrets constantly. Permissions apply: the caller needs read rights on the queried objects. The result can be large, and logic depending on it makes charts less deterministic and harder to test. I use lookup sparingly, always handle the empty case, and prefer explicit values or external secret management for stability.

Likely follow-ups

- What does lookup return in helm template?
- Why is it risky with Argo CD?

Lookup reads live cluster objects during rendering, for example to keep an existing secret. It returns empty offline, so helm template, diff and Argo CD render differently; it needs read RBAC and makes output nondeterministic. Always handle empty results and use it sparingly.

```
{{- $s := lookup "v1" "Secret" .Release.Namespace "db-pass" }}
{{- if $s }}
password: {{ index $s.data "password" }}
{{- else }}
password: {{ randAlphaNum 24 | b64enc }}
{{- end }}
```

Interview tip: Say lookup is empty offline; handle both cases.

📚 Start here (official docs)

- Helm documentation

certs.eknathalabs.com

Q92 How do you generate stable random values, such as passwords, in Helm without changing them on every upgrade?

Hooks/tests/advanced

🎙 Spoken answer · ~78 s

Template functions like randAlphaNum produce a new value every time the chart renders, so a secret generated that way changes on every upgrade, which can break applications and databases. Several patterns avoid it. One is lookup: check whether the secret already exists in the cluster and reuse its value, generating a new one only on first install, with the caveat that lookup fails to work offline and in GitOps renderers that do not contact the cluster. Another is requiring users to supply the value, or referencing a secret created elsewhere, which is the most robust and keeps generation outside Helm. A third is deriving a deterministic value from stable inputs with a hash function and a secret salt, such as sha256 of release name and a seed value, which is stable but only as secure as the seed. A pre-install hook job can create the secret once, though hook resources sit outside the release. Cluster-side tools such as External Secrets generators or password generators in operators create values once and manage rotation. I prefer external creation or lookup with care and avoid generating in plain templates.

Likely follow-ups

- Why is randAlphaNum dangerous in a template?
- What is the weakness of a deterministic hash approach?

Random functions regenerate on every render. Keep secrets stable by reusing existing objects with lookup, requiring supplied values, creating them with a one-time hook, or generating them outside Helm with External Secrets or operators. Deterministic hashes are stable but only as strong as their seed.

```
helm upgrade --install db ./chart --set auth.existingSecret=db-credentials
```

Interview tip: Say never leave randAlphaNum in an upgradeable secret.

📚 Official docs

- Helm: Chart template guide
- Helm: Values
- Helm: Upgrade

certs.eknathalabs.com

Q93 What are the Helm SDK and Helm in Go or Python automation, and when would you use them instead of the CLI?

Hooks/tests/advanced

🎙 Spoken answer · ~78 s

Helm is written in Go and its packages are importable as the Helm SDK, giving programmatic access to loading charts, rendering templates, installing, upgrading, rolling back and listing releases with the same behaviour as the CLI. Controllers and platform tools use it, for example Flux's helm-controller, to run Helm actions from within a service, with typed configuration instead of shelling out and parsing text. It is suited to building operators, deployment platforms or services that manage many releases, where error handling, concurrency, custom logging and integration with other systems matter. For scripting and pipelines, the CLI remains simpler and more portable, with machine-readable output through the output json flag and exit codes. In Python or other languages, options are calling the CLI through subprocess with explicit argument lists and timeouts, parsing JSON output, or using thin wrappers around the CLI or the Go library; there is no official Python SDK. Considerations include Helm version compatibility between the SDK and the target releases, managing kubeconfig and credentials safely, and testing. I use the CLI by default and the SDK only when building a long-running controller or service.

Likely follow-ups

- What does Flux use internally?
- How do you automate the CLI safely from Python?

The Helm SDK is a Go library exposing install, upgrade, rollback and template actions, used by controllers like Flux. For scripts the CLI with JSON output and exit codes is simpler; from Python call it with argument lists and timeouts. Use the SDK for long-running platform services.

```
helm list -A -o json | jq '.[] | {name, namespace, status, chart}'
```

```
subprocess.run(['helm','upgrade','--install',rel,chart,'-n',ns,'--atomic'], check=True,
timeout=900)
```

Interview tip: Say CLI for pipelines, SDK for controllers.

📚 Official docs

- Helm documentation
- Python Documentation

certs.eknathalabs.com

Q94 How would you write unit tests for chart templates with helm-unittest?

Hooks/tests/advanced

🎙 Spoken answer · ~74 s

Helm-unittest runs tests defined in YAML files placed in the chart's tests directory. Each suite names the template files it covers, optionally sets values or value files and release settings, and lists test cases with assertions on the rendered documents. Assertions include equal and notEqual for a path such as the container image, matchRegex, contains, exists, isKind, hasDocuments to count documents, failedTemplate to check that a required value triggers the expected error message, and matchSnapshot to compare against stored output. A typical set verifies that the default values produce expected resources, that enabling ingress creates an ingress with the right host, that disabling autoscaling renders replicas and enabling it omits them, that security defaults are present, that required values fail with a clear message, and that labels and selectors match. Tests run without a cluster, in milliseconds, in CI on every pull request. They complement lint, schema validation, kubeconform and install tests rather than replace them. Snapshots need review to avoid blindly accepting changes, and tests should cover edge cases found in past incidents so regressions are prevented.

Likely follow-ups

- How do you test that a required value fails?
- What are the risks of snapshot tests?

helm-unittest suites in the tests directory set values and assert on rendered documents with equal, contains, isKind, hasDocuments, failedTemplate and snapshots. They run fast without a cluster; cover defaults, toggles, security settings and required-value errors, and complement install tests.

```
suite: deployment
templates: [deployment.yaml]
tests:
  - it: omits replicas when autoscaling
    set: {autoscaling.enabled: true}
    asserts:
      - notExists: {path: spec.replicas}
```

```
helm unittest ./chart
```

Interview tip: Say test toggles and failure cases, not only defaults.

📚 Official docs

- Helm: Chart template guide
- Helm: Plugins
- Helm: Using Helm / commands

certs.eknathalabs.com

Q95 How do you manage Helm releases with Helmfile or similar tools for many releases and environments?

Hooks/tests/advanced

🎙 Spoken answer · ~71 s

Running individual helm commands for dozens of releases becomes unmanageable, so declarative tools describe the whole set. Helmfile uses a YAML file, with Go templating support, listing repositories, releases, chart references, versions, namespaces, values files and environment-specific settings. A single helmfile apply computes diffs with the diff plugin and installs or upgrades only the releases that changed, in dependency order defined with needs, and helmfile template, lint and diff support CI review. Environments are modelled with layered values and environment blocks, so production, staging and regions share definitions with small overrides. It can decrypt SOPS-encrypted values and run hooks. Benefits are a single reviewable source of truth, repeatable cluster bootstrap and consistent ordering. Drawbacks are another tool and templating layer to learn, and push-based deployment, which lacks continuous drift correction. Alternatives are GitOps controllers like Flux and Argo CD, which reconcile continuously and are generally preferred for production, with Helmfile still useful for bootstrapping clusters, local development and pipelines that cannot run controllers. I choose by team practice and compliance needs.

Likely follow-ups

- How does Helmfile order releases?
- Helmfile versus Flux or Argo CD?

Helmfile declares repositories, releases, versions and per-environment values in one file, applies only changed releases with diffs and dependency ordering via needs. It suits bootstrapping and push pipelines; GitOps controllers add continuous reconciliation and are preferred for production.

```
releases:
  - name: ingress
    namespace: ingress
    chart: ingress-nginx/ingress-nginx
    version: 4.11.3
    values: [values/ingress.yaml]
```

```
helmfile -e prod diff && helmfile -e prod apply
```

Interview tip: Say Helmfile is declarative push; GitOps is declarative pull.

📚 Official docs

- Helm: Chart hooks
- Helm: Chart template guide
- Helmfile

certs.eknathalabs.com

Q96 How do you design charts for stateful applications such as databases, and what are the pitfalls?

Hooks/tests/advanced

🎙 **Spoken answer · ~75 s**

Stateful workloads need careful handling because data outlives the release. The chart usually uses a StatefulSet with stable network identities, a headless service and volume claim templates for per-replica storage. Pitfalls include immutable volume claim template fields, so storage class or size changes require special procedures, expansion needing a storage class that allows it; PVCs persisting after uninstall, which protects data but leaks cost; ordered startup and update behaviour; and leaving credentials generated per render. Upgrades must handle schema or version compatibility, take backups before major changes, and use partition or on-delete update strategies for controlled rollouts. Pod disruption budgets, anti-affinity and topology spread protect availability. Readiness should reflect replication state, not just process liveness, and termination grace periods allow clean shutdown. Many production databases are better managed with operators, which understand backup, failover, scaling and upgrades, with the chart deploying the operator and the custom resource describing the cluster. Embedding a database as a subchart is acceptable for development and tests, but not for production data where lifecycle, backups and ownership need explicit treatment. Restore procedures are tested.

Likely follow-ups

- Why prefer operators for production databases?
- What happens to PVCs when a release is uninstalled?

Use StatefulSets with headless services, volume claim templates, PDBs, spread and meaningful readiness, but expect immutable template fields, persistent PVCs, credential stability and upgrade compatibility issues. Prefer operators for production databases, back up before upgrades and test restores.

```
volumeClaimTemplates:
  - metadata: {name: data}
    spec:
      accessModes: [ReadWriteOnce]
      resources: {requests: {storage: {{ .Values.persistence.size }}}}
```

```
kubectl get pvc -n prod
```

Interview tip: Say data outlives the release; plan for it.

📚 **Official docs**

- Helm: Provenance and integrity
- Helm documentation

certs.eknathalabs.com

Q97 How would you migrate from raw manifests or Kustomize to Helm, or from Helm to GitOps-native tooling, with minimal risk?

Hooks/tests/advanced

🎙 Spoken answer · ~73 s

Migration should be gradual and reversible. From raw manifests to Helm, I begin by converting one application: extract the repeating parts into templates and values, render the chart and diff it against the current live objects until the output is equivalent, ideally identical apart from labels Helm adds. Then adopt the existing resources into a release by adding Helm ownership labels and annotations, so the first install takes over without recreating pods, or install alongside and cut traffic over, and keep the old manifests until confidence is established. Immutable fields and selectors must be preserved exactly. From Helm to GitOps-native tooling, such as Argo CD or Flux, point the controller at the same chart and values in Git; with Flux's helm-controller the existing release can be taken over with matching release name and namespace, while Argo CD re-renders and applies, so I check hooks, lookups and ownership first and disable manual Helm use afterwards. Throughout I use diffs, staging rehearsals, backups, and a rollback plan, migrate low-risk services first and document the steps for teams.

Likely follow-ups

- How do you take over existing resources without downtime?
- What changes when Argo CD replaces helm install?

Convert one app at a time, render and diff until output matches live objects, preserve selectors, adopt resources via Helm labels and annotations or run in parallel then cut over. For GitOps, point controllers at the same chart and values, check hooks and lookups, rehearse in staging with backups.

```
helm template app ./chart -f values.yaml | kubectl diff -f -
```

```
kubectl annotate deploy app meta.helm.sh/release-name=app meta.helm.sh/release-
namespace=prod --overwrite
```

Interview tip: Say diff until it's boring, then adopt.

📚 Official docs

- Kusto Query Language (KQL)
- OpenGitOps principles
- Helm: Chart template guide

certs.eknathalabs.com

Q98 What are common Helm anti-patterns and best practices you would enforce in code review?

Hooks/tests/advanced

🎙 Spoken answer · ~77 s

Anti-patterns I flag include hard-coded values and names in templates; deep logic and long conditionals that make templates unreadable; forking third-party charts instead of customising through values or post-renderers; floating versions such as latest tags or unpinned dependency ranges; storing secrets in values files; using reuse-values or ad-hoc set flags in production so state cannot be reproduced; changing selectors or resource names between versions; missing resource requests, probes and security contexts; huge umbrella charts that couple unrelated services; hooks used for long-running or non-idempotent work; no schema, tests or documentation; and publishing charts without bumping versions. Best practices are the opposite: follow the recommended label conventions and the fullname helper; make values a documented, validated interface with a schema and safe defaults; keep templates simple, factoring helpers; pin chart and image versions, preferably digests; run lint, kubeconform, policy checks, unit and install tests, and helm diff in CI; deploy with atomic, wait and timeouts; keep values and chart versions in Git with GitOps; handle CRDs deliberately; and write upgrade notes for breaking changes. A short checklist in the pull request template turns these into habit.

Likely follow-ups

- Why is forking upstream charts discouraged?
- Which flags make production deploys safer?

Avoid hard-coded values, complex templates, forks, floating versions, secrets in values, reuse-values, unstable selectors, missing limits and probes, hooks for heavy work and untested charts. Enforce schema, tests, diffs, pinned versions and digests, atomic deploys and GitOps, with a review checklist.

```
helm lint . --strict && helm unittest . && helm template . | kubeconform -strict
```

```
helm upgrade --install app . --atomic --wait --timeout 10m
```

Interview tip: Say simple templates, pinned versions, tested upgrades.

📚 Official docs

- Microsoft: Anti-patterns (performance antipatterns)
- Helm: Chart best practices
- Helm documentation

certs.eknathalabs.com

Q99 How do you handle breaking changes when upgrading a major chart version, for example a third-party chart that renamed values?

Hooks/tests/advanced

🎙 Spoken answer · ~78 s

I treat a major chart upgrade as a small project. First read the changelog and upgrade notes for removed or renamed values, changed defaults, new minimum Kubernetes or dependency versions, CRD changes and required manual steps. Then fetch both chart versions and diff the default values and templates to see everything that changes. In a staging environment that mirrors production, back up data, and run the upgrade with my existing values plus the diff plugin, which shows the real Kubernetes-level changes: selector changes that will fail as immutable, recreated stateful sets, removed resources, and changed images. Update my values files to the new keys, using the schema or lint errors to catch leftovers, and apply any manual steps such as CRD upgrades through kubectl first. Where it is risky, use a maintenance window, a blue-green approach by installing the new chart version alongside the old one and migrating data, or step through intermediate versions if the notes require it. Verify with smoke tests and metrics, and keep a rollback plan, remembering data migrations may not be reversible. Document the outcome for other environments, then promote in order.

Likely follow-ups

- Why diff both chart versions?
- When use blue-green for a major chart upgrade?

Read upgrade notes, diff default values and templates between versions, rehearse in staging with backups and helm diff, update renamed values, apply CRD steps manually, consider intermediate versions or blue-green for risky changes, verify with tests and promote with a rollback plan.

```
helm pull repo/chart --version 14.0.0 --untar --untardir new && helm pull repo/chart --
version 13.5.2 --untar --untardir old && diff -ru old new | less
```

```
helm diff upgrade app repo/chart --version 14.0.0 -f values.yaml
```

Interview tip: Say rehearse in staging with diff and backups.

📚 Official docs

- Google API Design Guide
- Helm: Values
- Artifact Hub

certs.eknathalabs.com

Q100 How do you build and use Helm OCI dependencies and chart caching in air-gapped or restricted environments?

Hooks/tests/advanced

🎙 Spoken answer · ~76 s

Air-gapped clusters cannot reach public registries, so everything is mirrored and resolved internally. The process starts in a connected staging area where approved charts and images are pulled, scanned and reviewed, then copied into an internal OCI registry with tools such as helm pull and helm push, oras, skopeo or crane, or a registry replication feature in Harbor, preserving versions and digests. Chart dependencies in Chart.yaml point at the internal registry, so dependency build resolves offline, and Chart.lock keeps exact versions. Image references in values are rewritten to the internal registry, using global image registry values, or a post-renderer, or a mirror configured on the container runtime. For a fully disconnected network, charts and images are exported into a signed bundle, transferred over an approved medium, verified against signatures and digests, then imported. Vendoring dependency archives in Git removes registry reliance for builds. The Helm cache and config directories in CI are set to known paths, so runs are reproducible without network calls. Updates follow a regular cadence with the same scan and approval steps, and an inventory records which versions are present.

Likely follow-ups

- How do images get rewritten to the internal registry?
- What verifies a transferred bundle?

Mirror reviewed charts and images into an internal OCI registry, point dependencies and image values at it, keep Chart.lock and vendored archives for offline builds, transfer signed bundles for fully disconnected networks, verify digests and signatures and refresh on a regular approved cadence.

```
helm pull oci://public.example/charts/app --version 1.4.0 && helm push app-1.4.0.tgz
oci://registry.internal/charts
```

```
skopeo copy docker://public/app:1.4.0 docker://registry.internal/app:1.4.0
```

Interview tip: Say mirror, pin, verify, rewrite image registries.

📚 Official docs

- Helm: Registries (OCI)
- Helm: Charts and dependencies
- Helm documentation

certs.eknathalabs.com